Real-Time Multi-Core Sorting and Median Filter Service

A Report

Submitted for the course

26ELC331 Real Time Operating System using QNX

by

Anmol Govindarajapuram Krishnan (CB.EN.U4EEE23103)

Mauli Rajguru (CB.EN.U4EEE23120)

Sanjay TS (CB.EN.U4EEE23136)



DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

AMRITA VISHWA VIDYAPEETHAM

COIMBATORE CAMPUS (INDIA)

October 17, 2026

Contents

List of Figures **iv**

List of Tables **vii**

Abbreviations **viii**

List of Symbols **ix**

1 Introduction **1**

1.1 Background and Motivation 1

1.2 Project Objectives 1

1.3 Workload and Dataset 2

1.4 Report Structure 3

2 System Overview **4**

2.1 Introduction 4

2.2 System Architecture and Block Diagram 4

2.2.1 Hardware Setup 6

2.3 Client-Server Communication Sequence 6

2.4 Data Flow Diagram 10

3 Methodology **12**

3.1 Introduction 12

3.2 Virtual Sensor 12

3.2.1 Configuration 12

3.2.2 Addressing a sample in either layout 13

3.2.3 The read-ahead window 13

3.2.4 Frames, blocks and the sensor's clock 14

3.2.5 Rejecting bad input, and the interface . . . 15
3.3 Block Loader . . . 15
3.3.1 Handling Multiple File Layouts . . . 16
3.3.2 Zero-Padding and the Descriptor . . . 17
3.4 Parallel Merge-Sort . . . 18
3.4.1 The pool and the plan . . . 18
3.4.2 The merge, and why the answer is stable . . . 18
3.4.3 The barrier between phases . . . 20
3.4.4 Refusals, and the plan line . . . 20
3.4.5 Measured scaling . . . 21
3.5 Rolling Median Filter (Hampel Filter) . . . 21
3.5.1 Window-Based Median and MAD Calculation . . . 22
3.5.2 Filter Configuration and Boundary Handling . . . 22
3.6 Client Implementation . . . 23
3.6.1 Connection Deadline . . . 23
3.6.2 Pipeline Orchestration and Block Preparation . . . 24
3.6.3 Two-Part IOV Submission . . . 24
3.6.4 Completion Pulse Handling . . . 26
3.6.5 Shared Memory Retrieval . . . 26
3.6.6 Final Verification and Output . . . 26
3.7 Server Implementation . . . 27
3.7.1 Initialization and Resource Allocation . . . 27
3.7.2 Two-Stage Reception and Validation . . . 27
3.7.3 Early Acknowledgement . . . 28
3.7.4 Block Processing and Function Calls . . . 28
3.7.5 Global Run Aggregation . . . 29
3.7.6 Asynchronous Pulse Notification . . . 30
3.8 QNX IPC Mechanisms . . . 30
3.9 End-to-End Pipeline Execution . . . 31
3.9.1 Phase 1: Local Preparation and Submission . . . 31
3.9.2 Phase 2: Computation and Publication . . . 32
3.9.3 Phase 3: Retrieval and Post-Processing . . . 33

3.10 Testing . . . 33
3.10.1 Two folds, one set of sources . . . 34
3.10.2 Four kinds of evidence . . . 35
3.10.3 The suites and what they establish . . . 36
3.10.4 The board run . . . 37

4 Results and Discussions **43**
4.1 Experimental Setup . . . 43
4.2 Filter Quality and Outlier Removal (Width Sweep) . . . 43
4.2.1 Overall Filter Quality . . . 43
4.3 Channel-Wise Filtering Results . . . 46
4.3.1 Voltage Channel . . . 46
4.3.2 Current Channel . . . 46
4.3.3 Temperature Channel . . . 49
4.4 Processing Cost and Timing Analysis . . . 51
4.5 Parallel Sorting Performance . . . 52
4.6 System Overhead and IPC . . . 53
4.6.1 QNX Board Execution and Validation . . . 53

5 Conclusion and Scope for Further Research **56**
5.1 Practical Industrial Applications . . . 56
5.2 Conclusion . . . 57
5.3 Scope for Further Research . . . 57

List of Figures

2.1 System architecture: the client subsystem, the QNX IPC paths, the shared results region and the server subsystem. 5
2.2 Hardware setup showing the Raspberry Pi GPIO header and the connected FTDI interface board. 6
2.3 Client subsystem: the interleaved recording, the virtual sensor, the block assembler and the client controller. 8
2.4 Server subsystem: the IPC request dispatcher, the parallel merge-sort engine and the rolling median filter. 9
2.5 Data flow through the service: from the recording on disk, through the sensor and block loader, across the message bus, into the shared results region. 11

3.1 How one sample is addressed. A planar file holds one sample per float, so sample i is float i. An interleaved file holds four floats per record, so sample i of the voltage channel is float $4i + 1$; the other three floats of each record are skipped. 13
3.2 The sliding window. Everything behind it has been delivered, everything ahead has not been read; when `next` reaches the window's end, one `fread` refills the window from the new position. 14
3.3 The plan for one 64 KiB block on four workers. Phase 0 sorts four partitions, each into its own slice of the scratch buffer; each later phase merges adjacent runs and writes the other buffer, so the answer alternates between the caller's array and the scratch. A phase cannot begin until every task of the previous one has finished the dashed barriers. 19
3.4 Execution flow of the Client. The client loops through the blocks, utilizing QNX message passing. Once the recording ends (EOF), it performs a global sort, filters the data, writes the output files, and exits. 25

3.5 End-to-end execution of a 64 KiB data block. Message passing handles requests and control signals, shared memory stores the processed arrays, and QNX pulses notify the client when processing is complete. 32
3.6 The protocol suite on the board: the wire contract, the shared-region layout and the seal's sensitivity, 186 checks. 37
3.7 The sensor suite on the board. The `sensor:` lines are the test failing on purpose (a missing file, a 3-byte file, a 6-float record), each followed by `ok`. 38
3.8 The block loader suite on the board: the interleaved recording produces the same blocks and the same seals as the host. 39
3.9 The merge sort suite on the board: ordering, the multiset, the plan line and byte-identical output across thread counts. 39
3.10 The median suite on the board, the largest at 7 615 checks: the filter against a reference, across seams, bursts and dropouts. 40
3.11 The shared-memory suite: the results region's bounds and arithmetic, 13 checks. 40
3.12 The KPI benchmark on the board: sort throughput at one, two and four workers, and the global median computed two ways agreeing bit for bit. 41
3.13 The client deadline test: with no server the client gives up at ten seconds, and with a server that starts three seconds late it connects by itself. 41
3.14 The soak: twenty consecutive runs, every one completing with identical medians and a stable service time per block. 42

4.1 Filter quality metrics for different rolling-window widths. 44
4.2 Error map comparing the raw and filtered sensor data against the reference signal. 45
4.3 Raw and filtered voltage signals using the rolling Hampel filter. 46
4.4 Zoomed view of the raw and filtered voltage signals, highlighting the local effect of Hampel filtering. 47
4.5 Filtering error for the voltage channel. 47
4.6 Raw and filtered current signals using the rolling Hampel filter. 48
4.7 Zoomed view of the raw and filtered current signals, highlighting the local effect of Hampel filtering. 48
4.8 Filtering error for the current channel. 49
4.9 Raw and filtered temperature signals using the rolling Hampel filter. 49

4.10 Zoomed view of the raw and filtered temperature signals, highlighting the local effect of Hampel filtering. 50
4.11 Filtering error for the temperature channel. 50
4.12 Measured sorting, filtering and total server processing time as a function of rolling-window width. 51
4.13 Trade-off between filtering quality and computational cost for the voltage channel. 52
4.14 Measured processing cost and speedup as the number of merge-sort worker threads increases. 52
4.15 Distribution of processing time across pipeline stages for different merge-sort worker counts. 53
4.16 Boot evidence from the QNX target environment. 54
4.17 Completion of the processing pipeline on the QNX target. 54

List of Tables

3.1 Fields of `sensor_config_t`. Passing a null configuration to `sensor_open` selects the defaults in the last column. 12
3.2 The sensor's interface, including the counters that report the recording and the sensor's own timeline. 15
3.3 The sort alone, per 64 KiB block, against the number of workers. The plan column is what the run reported. 21
3.4 The kinds of evidence the suites use, and why each is convincing. 35
3.5 The six suites. The checks are per suite; the total is the figure quoted for both folds. The KPI suite is a benchmark and is counted separately. 36

4.1 Filter quality comparison for window widths of 9 and 15 samples. 45
4.2 Measured average timing of the QNX processing pipeline for the voltage channel at filter width 15. 55

Abbreviations

AWGN Additive White Gaussian Noise

BSP Board Support Package

FNV Fowler–Noll–Vo

IOV Input/Output Vector

IPC Inter-Process Communication

MAD Median Absolute Deviation

POSIX Portable Operating System Interface

QNX QNX Neutrino RTOS

RMSE Root Mean Square Error

RTOS Real-Time Operating System

SPM Single Particle Model

List of Symbols

σ Standard deviation (noise parameter)

w Filter width (window size in samples)

f_s Sampling frequency (Hz)

N Number of samples / block size

k Number of partitions / worker threads

Chapter 1

Introduction

1.1 Background and Motivation

Embedded systems used for real-time data acquisition often need to process large amounts of sensor data while meeting strict timing requirements. Multi-core processors help meet these demands by allowing computational tasks to run in parallel. However, making effective use of multiple cores requires proper task distribution and synchronization, especially in systems where predictable execution time and responsiveness are important.

Sensor data may also contain unwanted variations, such as sudden spikes, bursts of abnormal readings, or dropouts caused by electromagnetic interference, sensor faults, or changes in operating conditions. Identifying and removing these outliers is important for obtaining reliable results during statistical analysis. Combining this filtering process with data sorting increases the computational workload. In a Real-Time Operating System (RTOS), these operations must be handled efficiently without affecting the system's ability to respond to inter-process communication (IPC) requests and hardware interrupts.

1.2 Project Objectives

This project focuses on developing a real-time client-server system using the QNX RTOS and deploying it on a Raspberry Pi 4. The system is designed to transfer unsorted floating-point time-series data from a client to a server in blocks of at least 64 KB for processing. The server performs two main operations:

1. **Parallel Sorting:** A parallel Merge-Sort algorithm is implemented using multiple POSIX worker threads. Mutexes and condition variables are used to coordinate thread execution and synchronization during the sorting and merging stages.

2. **Rolling Median Filtering:** A robust rolling median filter (Hampel filter) is applied to identify and remove outliers from the time-series data. The filter uses local median and median absolute deviation (MAD) values to detect abnormal readings while preserving

the underlying signal.

After processing, the server notifies the client through a QNX pulse event containing the calculated median and a descriptor specifying the relevant memory offset. The processed arrays remain in shared memory, allowing the client to access the results without requiring another complete data transfer.

1.3 Workload and Dataset

The system is evaluated using a synthetic dataset generated offline with PyBaMM, a Python-based battery modelling package. The dataset contains battery measurements, including current, voltage, and temperature, representing a simulated discharge cycle. To evaluate the ability of the filtering algorithm to detect abnormal readings, controlled noise and outliers are introduced into the data.

The dataset includes the following types of corruption:

- **Gaussian Noise:** Additive white Gaussian noise is introduced into the measurements to simulate variations in sensor readings. Example noise levels include 2 mV for voltage and 8 mA for current.
- **Impulsive Outliers:** Outliers are introduced in the form of sudden spikes, consecutive bursts, and stuck readings. These outliers are introduced at random and occur at a rate of 1.5%. A ground-truth mask is also maintained to identify the injected outliers and evaluate the filtering performance.

This dataset provides a controlled environment for evaluating the sorting and filtering algorithms under different data conditions. It also allows the results to be compared against the known locations of the injected outliers.

1.4 Report Structure

The remainder of this report is organized into four chapters:

- **Chapter 2 (System Overview)** presents the system architecture, including the block diagram, data flow, and sequence of messages exchanged between the client and server.
- **Chapter 3 (Process Implementation)** explains the implementation of the parallel Merge-Sort algorithm, the rolling median filter, and the QNX IPC mechanisms used for communication and data sharing, including IOV transfers, pulses, and shared memory.
- **Chapter 4 (Results and Discussions)** evaluates the system's performance through thread-scaling experiments, filtering accuracy measurements, and execution-time analysis on the Raspberry Pi 4.
- **Chapter 5 (Conclusion)** summarizes the project outcomes and discusses possible improvements and directions for future work.

Chapter 2

System Overview

2.1 Introduction

This chapter provides a high-level overview of the system architecture. It describes the interaction between the client and server, the flow of data, and the communication mechanisms utilized within the QNX RTOS.

2.2 System Architecture and Block Diagram

The system is based on a client-server model running on QNX. The client reads data from a virtual sensor and offloads it in 64 KB blocks to the server. The server utilizes a multi-threaded approach to sort the data and apply a Hampel filter. The results are written to a shared memory region.

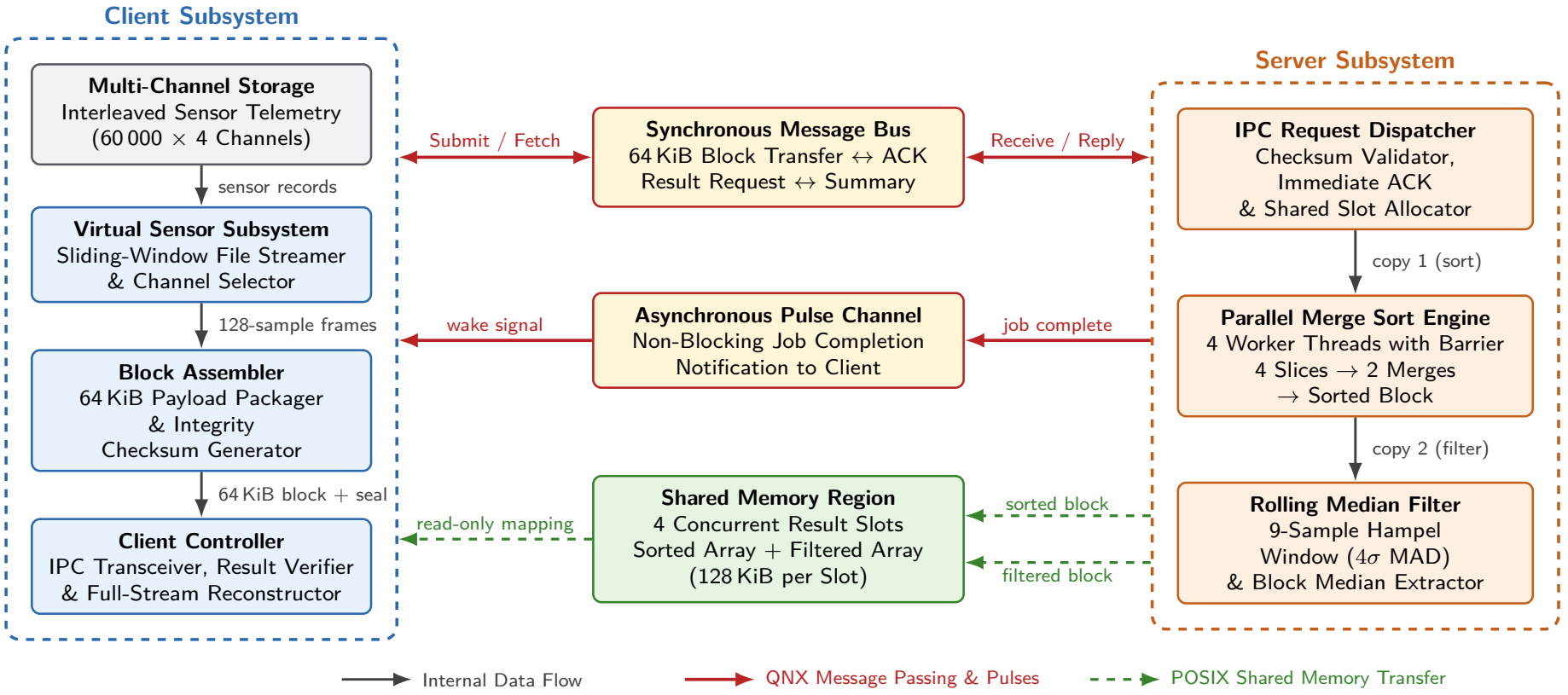


Figure 2.1: System architecture: the client subsystem, the QNX IPC paths, the shared results region and the server subsystem.

2.2.1 Hardware Setup

The system was deployed on a Raspberry Pi 4 running QNX Neutrino RTOS. The hardware setup diagram illustrates the Raspberry Pi GPIO connections to the external FTDI-based interface board used in the setup. The Raspberry Pi provides the target computing platform on which the client-server application and its processing pipeline execute.

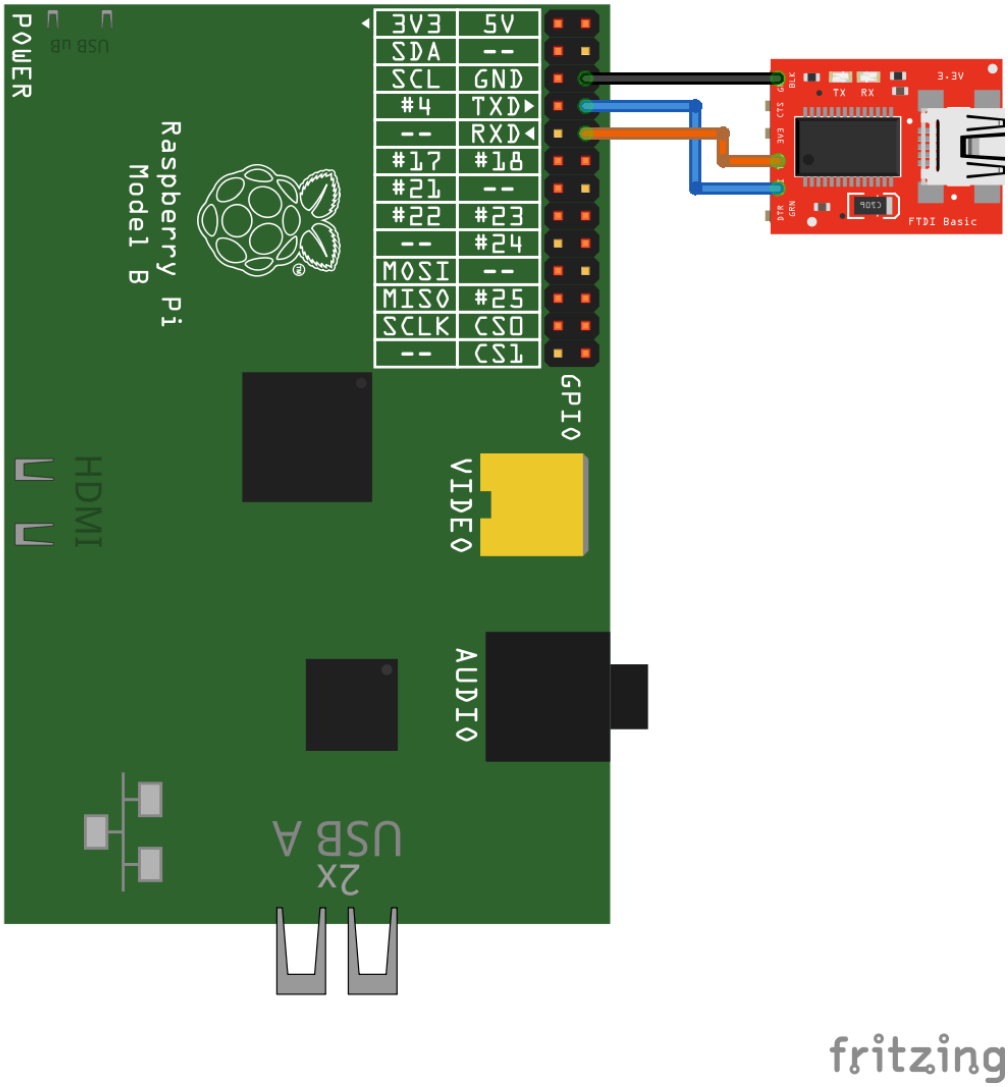


Figure 2.2: Hardware setup showing the Raspberry Pi GPIO header and the connected FTDI interface board.

2.3 Client-Server Communication Sequence

Communication between the two processes follows a strict sequence to ensure data integrity and asynchronous notification.

1. The client connects to the server using `name_open`.

2. The client sends a 64 KB block using an IOV message (`MsgSendv`).

3. The server validates the block, sends an acknowledgement, and queues the job.

4. Once processing is complete, the server sends a QNX pulse back to the client.

5. The client retrieves the result descriptor using a fetch message.

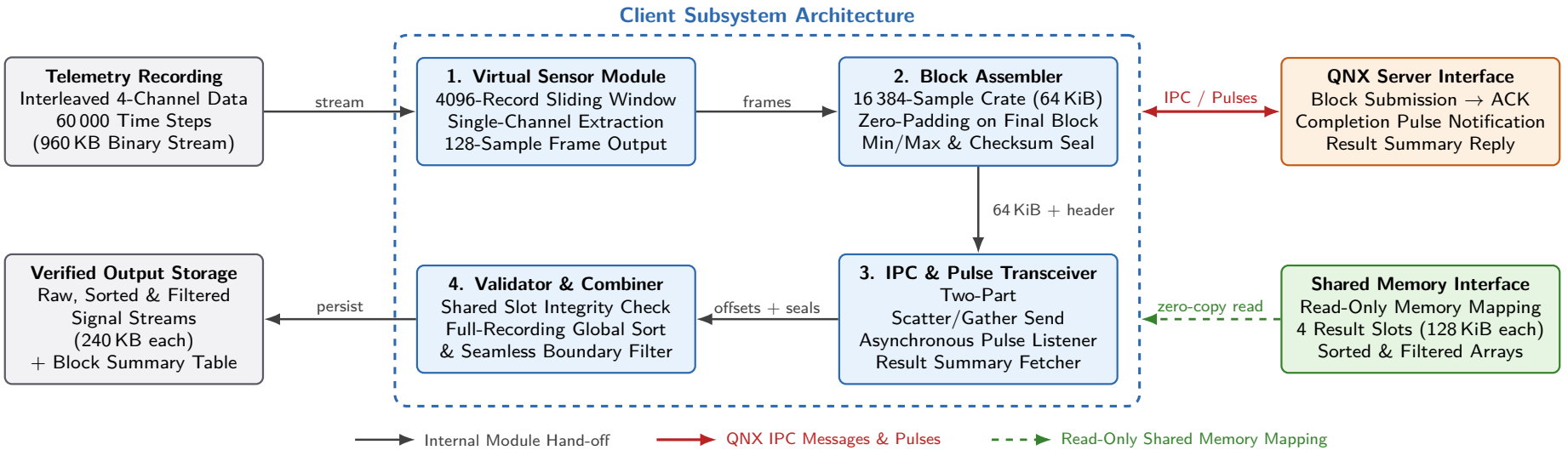


Figure 2.3: Client subsystem: the interleaved recording, the virtual sensor, the block assembler and the client controller.

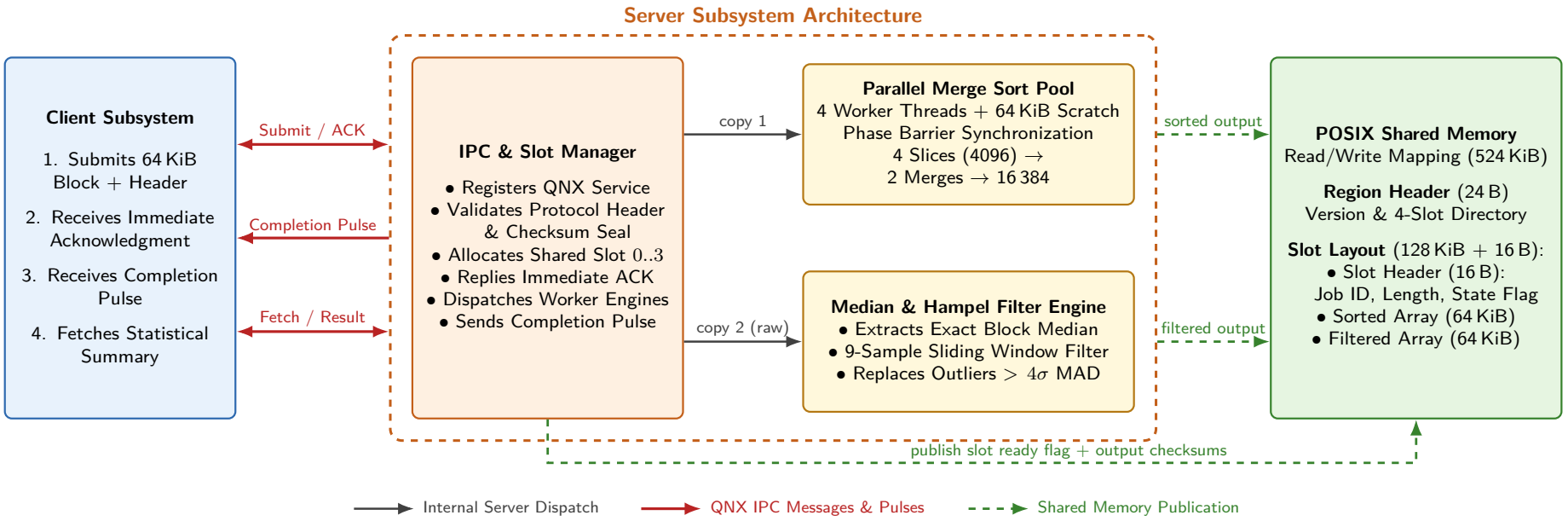


Figure 2.4: Server subsystem: the IPC request dispatcher, the parallel merge-sort engine and the rolling median filter.

2.4 Data Flow Diagram

The data starts as raw binary files on disk, is loaded by the client, transferred over the QNX message passing kernel, sorted and filtered, and finally written into a mapped shared memory region.

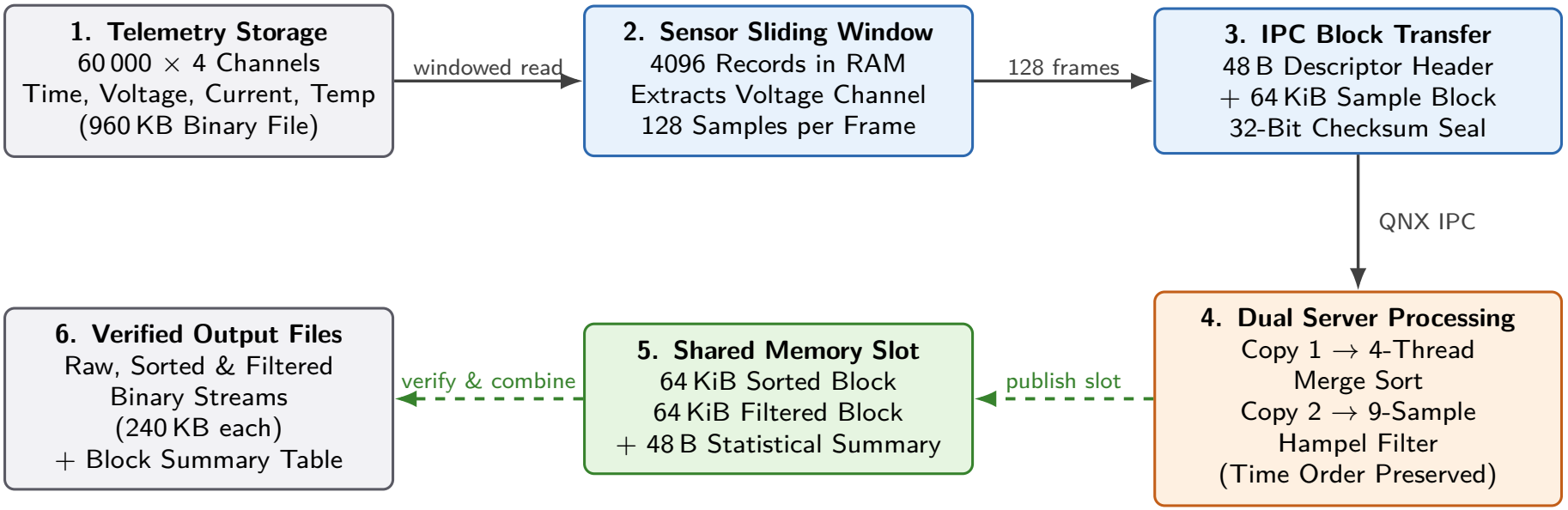


Figure 2.5: Data flow through the service: from the recording on disk, through the sensor and block loader, across the message bus, into the shared results region.

Chapter 3

Methodology

3.1 Introduction

This chapter details the implementation of the core algorithms, the data structures, and the QNX-specific mechanisms used to build the service.

3.2 Virtual Sensor

There is no physical sensor on the bench, so the service is fed by a module that replays a recording with the behaviour of a live device. Values arrive one sample at a time, in frames, on a clock the module keeps itself rather than at the speed of the host. The purpose is not convenience: with the sensor behind this interface the client and the rest of the pipeline can be developed and tested exactly as they will run against hardware, and a real device can replace the module later without changing anything downstream.

3.2.1 Configuration

Everything the caller can choose sits in one structure, so the two file layouts are configuration rather than separate code paths:

Table 3.1: Fields of `sensor_config_t`. Passing a null configuration to `sensor_open` selects the defaults in the last column.

Field	Type	Meaning	Default
`layout`	`int`	`SENSOR_LAYOUT_PLANAR`, one sample per float, or `SENSOR_LAYOUT_INTERLEAVED`, `stride` floats per record	planar
`offset`	`int`	interleaved only: which float of a record this sensor delivers	0
`stride`	`int`	interleaved only: floats per record	1
`frame_samples`	`size_t`	samples a single read may deliver	128
`rate_hz`	`uint32_t`	virtual rate used for timestamps and pacing	100
`loop`	`int`	restart at the end of the recording, as a live sensor would	0

3.2.2 Addressing a sample in either layout

Both layouts are headerless little-endian `float32`, and both are read by one expression, where `next` is the sample the sensor is about to deliver:

$$\texttt{value} = \texttt{window}\big[(\texttt{next} - \texttt{window_first}) \times \texttt{stride} + \texttt{offset}\big] \tag{3.1}$$

A planar file is `stride` 1 and `offset` 0, so sample i is float i. An interleaved file is four floats per record time, voltage, current, temperature so sample i of the voltage channel is float $4i + 1$: record 1 is float 5, byte 20 of the file. The three unused floats of each record are read from disk and discarded; a channel other than the selected one is never returned. Figure 3.1 shows both cases.

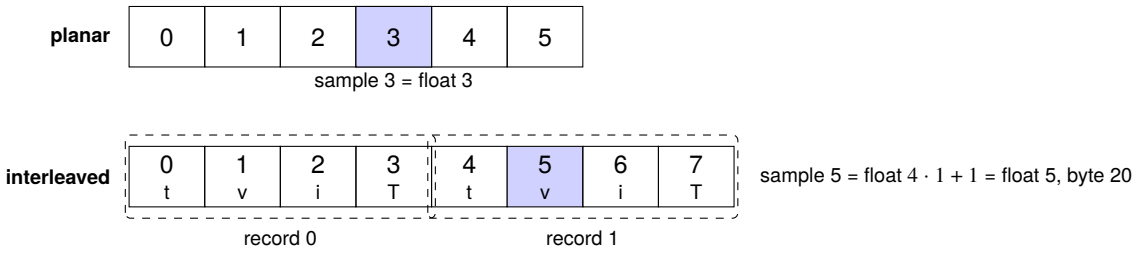


Figure 3.1: How one sample is addressed. A planar file holds one sample per float, so sample i is float i. An interleaved file holds four floats per record, so sample i of the voltage channel is float $4i + 1$; the other three floats of each record are skipped.

3.2.3 The read-ahead window

The recording is not loaded whole. Reads pass through a sliding window of `window_samples` records 4096 by default, widened when the caller's frame is larger which is refilled as the reader advances. In the real pipeline the block loader asks for block-sized frames, so the window becomes 16 384 records: 64 KiB of a planar file or 256 KiB of an interleaved one, in one call to `fread`. A caller reading one sample at a time therefore sees exactly the same values as a caller reading whole blocks, a property the tests use as an independent check.

The refill condition is the three-part test in Listing 3.1: the window is empty, the reader was rewound behind it, or it has walked past the end. The loop applies the test before every sample, so no case can read stale data.

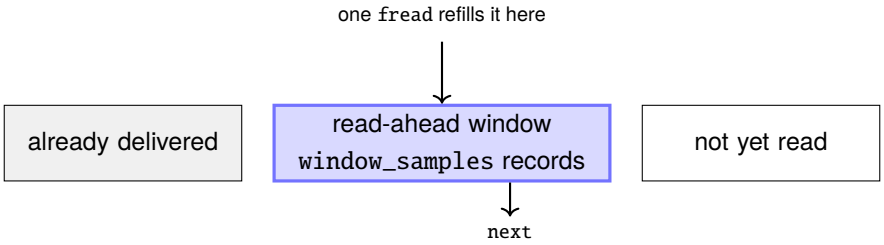


Figure 3.2: The sliding window. Everything behind it has been delivered, everything ahead has not been read; when `next` reaches the window's end, one `fread` refills the window from the new position.

Listing 3.1: The read loop of `sensor_read()`, trimmed of its error returns. A device read delivers at most one frame; aggregating frames into a block is the caller's job.

```
while (n < cap) {
    if (s->next >= s->total) {
        if (!s->cfg.loop)
            break; /* end of the recording */
        s->next = 0; /* a live sensor never stops */
    }

    if (s->window_count == 0 || /* nothing read yet */
        s->next < s->window_first || /* rewound behind the window */
        s->next >= s->window_first + s->window_count)
        sensor_fill(s, s->next); /* refill at the new position */

    offset = (s->next - s->window_first) * s->cfg.stride + s->cfg.offset;
    dst[n++] = s->window[offset]; /* the one channel that was asked for */
    s->next++;
    s->delivered++;
}
```

3.2.4 Frames, blocks and the sensor's clock

A device read returns a frame, not an arbitrary amount of data: `sensor_read` delivers at most `frame_samples` samples, and `sensor_read_frame` delivers exactly one frame except at the end of a recording. Assembling a 64 KiB payload is the client's concern, which is what `sensor_read_block` does: it calls `sensor_read` until the block is full, so the aggregation sits on the client side of the boundary rather than inside the device.

The clock is arithmetic rather than a system timer. The timestamp of the next sample is

`sensor_index` $\times 10^9 /$ `rate_hz` nanoseconds, computed from a counter of samples delivered, so after 600 samples at 100 Hz it is exactly 6 s however fast the host runs; in loop mode the counter keeps advancing across the wrap, as a sensor that never stops would. `sensor_read_paced` adds a wait after each read so the stream holds the configured rate. That wait and the monotonic clock behind it are the only platform-specific code in the project `clock_gettime` and `nanosleep` on QNX, with one shim for the host build and its resolution is a millisecond, so pacing is meaningful up to roughly 500 Hz.

3.2.5 Rejecting bad input, and the interface

Nothing is assumed about the file. A missing path, an empty file, a length that is not a whole number of `float32` values or of records, an offset outside the record, a stride below one, an unknown layout, a frame or rate out of range each returns a null handle or an error code and prints a sentence naming the problem instead of reading nonsense. Reads follow the same discipline: the number of samples delivered, 0 at the end of a recording, -1 for an error.

Table 3.2: The sensor's interface, including the counters that report the recording and the sensor's own timeline.

Function	Purpose
`sensor_total_samples(s)`	number of samples in the recording
`sensor_remaining(s)`	samples left before the end
`sensor_index(s)`	samples delivered since open or reset
`sensor_open(path, cfg)`	open a recording; a null configuration selects the defaults
`sensor_read(s, dst, max)`	deliver at most one frame
`sensor_read_frame(s, dst)`	deliver exactly one frame
`sensor_read_block(s, dst, n)`	aggregate frames into a block of n samples
`sensor_read_paced(s, dst, max)`	read, then wait for the configured rate
`sensor_timestamp_ns(s)`	the next sample's time on the sensor's clock
`sensor_reset(s)`	rewind and restart the timeline
`sensor_close(s)`	release the handle

3.3 Block Loader

The block loader connects the virtual sensor to the client by organizing the incoming sensor samples into fixed-size blocks for transmission. Each block contains 16,384 samples, as defined in the code using the name "`PROTO_BLOCK_SAMPLES`". Since each sample is a 32-bit floating-point value, the total block size is 65,536 bytes (64 KiB). The loader also prepares a descriptor

containing the metadata required to identify and validate each block.

3.3.1 Handling Multiple File Layouts

The dataset stores recordings in two formats: planar and interleaved. In the planar format, each channel is stored separately, whereas the interleaved format stores multiple channel values together in each record. The loader must account for these differences while ensuring that the correct data is selected for each protocol channel.

In the interleaved format, each record follows the order `[time_s, voltage_V, current_A, temperature_K]`. This order must be mapped correctly to the enumeration used by the protocol. To avoid selecting the wrong signal during processing, the loader handles this mapping through a dedicated function. For instance, voltage is stored at index 1, while current is stored at index 2.

Listing 3.2: Mapping protocol channels to interleaved file offsets using `block_loader_channel_offset`

```
int block_loader_channel_offset(proto_channel_t channel)
{
    switch (channel) {
    case PROTO_CHANNEL_TIME: return 0;
    case PROTO_CHANNEL_VOLTAGE: return 1; /* not 2: see the header note */
    case PROTO_CHANNEL_CURRENT: return 2;
    case PROTO_CHANNEL_TEMPERATURE: return 3;
    default: return -1;
    }
}
```

The loader also configures the virtual sensor's frame size to match the protocol block size using `cfg.frame_samples = PROTO_BLOCK_SAMPLES`. This allows `sensor_read_block()` to read the required 16,384 samples in a single operation for a full block, rather than processing each sample individually. This approach keeps the data-loading process straightforward and reduces the overhead associated with repeated read operations.

3.3.2 Zero-Padding and the Descriptor

The recording length may not always be an exact multiple of 16,384 samples. In such cases, the final block contains fewer valid samples than the protocol's fixed block capacity. Since the complete 64 KiB buffer is transmitted between processes through the QNX message-passing mechanism, the unused portion must be initialized before transmission. Otherwise, it could contain stale memory contents and produce inconsistent results between executions.

The loader sets all unused sample positions in the final block to zero, so the buffer is predictable even when the recording ends partway through a block. It then fills the block descriptor (sequence number, valid sample count, first sample index, rate, channel, and checksum). The checksum uses FNV-1a hash algorithm and covers only the valid samples, excluding the zero-padded tail.

Listing 3.3: Zero-padding the final block and populating its descriptor in `block_loader_next`

```
    /* Zero the unused tail of a short final block. */
    if ((uint32_t)got < PROTO_BLOCK_SAMPLES) {
        memset(&block->samples[got], 0,
               (size_t)(PROTO_BLOCK_SAMPLES - (uint32_t)got) * sizeof(float));
    }

    memset(desc, 0, sizeof(*desc));
    desc->sequence = (uint32_t)l->sequence;
    desc->samples = (uint32_t)got;
    desc->first_sample = l->samples;
    desc->rate_hz = l->rate_hz;
    desc->channel = (uint32_t)l->channel;
    desc->checksum = proto_checksum(block->samples,
                                    (size_t)got * sizeof(float));

    l->sequence++;
    l->samples += (uint64_t)got;
    return got;
}
```

3.4 Parallel Merge-Sort

Sorting a block is the service's one genuinely parallel computation, and it is the step where a mistake is hardest to see: a wrong sort still returns numbers of the right count. The implementation is therefore built around two properties that can be checked rather than argued. The output does not depend on how many workers ran, and the plan the workers actually executed is reported, so a run can be shown to have taken the parallel path rather than the serial one.

3.4.1 The pool and the plan

The threads are created once per process by `ms_pool_create` one per online CPU by default, four on the Raspberry Pi 4, capped at `MS_MAX_THREADS` (64) and they wait on a condition variable when there is nothing to do. Nothing is created or destroyed per block.

Each sort splits the one array it was given according to how much work a worker needs to be worth waking:

$$k = \min\left(\left\lfloor \frac{n}{1024} \right\rfloor, \texttt{nthreads}\right), \qquad \texttt{part} = \left\lceil \frac{n}{k} \right\rceil \tag{3.2}$$

so a 16 384-sample block on four workers becomes four partitions of 4096. Phase 0 posts one partition task per slice; each worker sorts its slice with the ordinary single-threaded merge sort, writing into its own region of one scratch buffer. What comes back is four sorted runs, not one sorted block. The merge phases then halve the number of runs merge adjacent pairs, swap which buffer is read and which is written, repeat until a single run covers the array. Figure 3.3 shows the shape of a block on four workers: three rounds, seven tasks.

3.4.2 The merge, and why the answer is stable

Two sorted runs become one by looking at the front of each and taking the smaller, which is the whole of the merge:

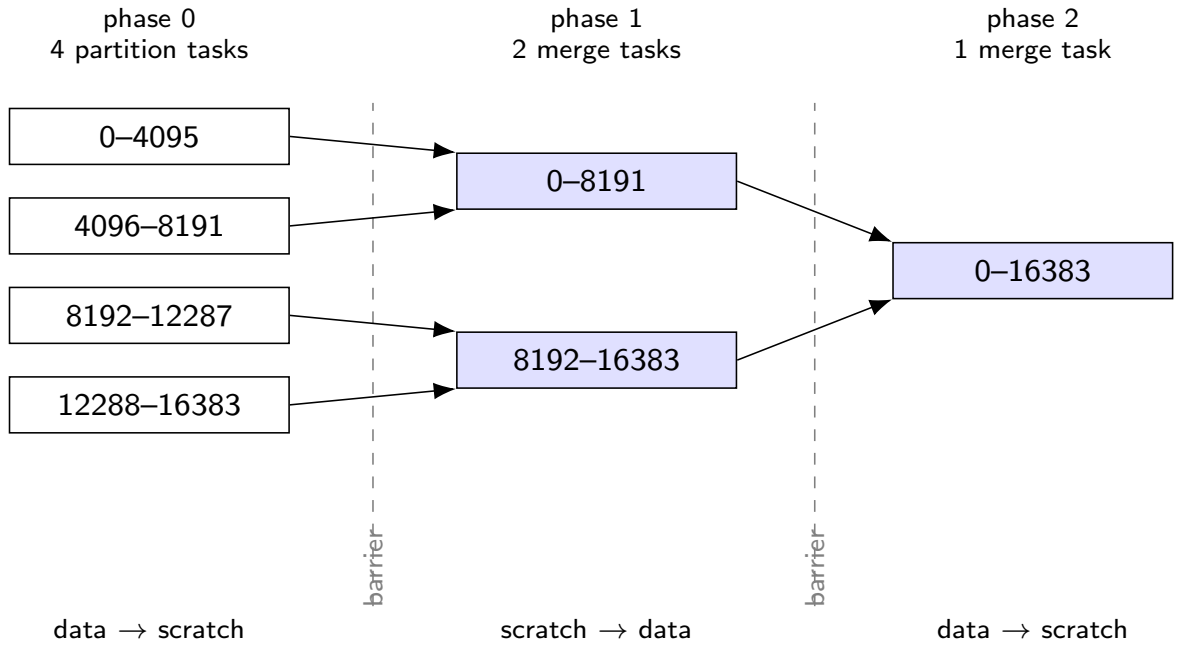


Figure 3.3: The plan for one 64 KiB block on four workers. Phase 0 sorts four partitions, each into its own slice of the scratch buffer; each later phase merges adjacent runs and writes the other buffer, so the answer alternates between the caller's array and the scratch. A phase cannot begin until every task of the previous one has finished the dashed barriers.

Listing 3.4: The merge step. The comparison is <=, so on a tie the value from the left run goes first.

```
static void merge_runs(const float *a, size_t na,
                       const float *b, size_t nb, float *out)
{
    size_t i = 0;
    size_t j = 0;
    size_t k = 0;

    while (i < na && j < nb)
        out[k++] = (a[i] <= b[j]) ? a[i++] : b[j++];
    while (i < na)
        out[k++] = a[i++];
    while (j < nb)
        out[k++] = b[j++];
}
```

That one character, <= rather than <, is what makes the sort *stable*: equal values keep the order in which they arrived. Stability is what makes the result independent of the thread count. A different partition count changes *when* two equal values meet, and the tie rule decides the same

way whichever side they come from, so one worker and eight workers produce byte-identical output. The test suite asserts exactly that, for one, two, three, five and eight workers.

3.4.3 The barrier between phases

Each phase's tasks go into one shared array and the workers claim them from a queue. The phase boundary is a barrier on completion, not a delay:

Listing 3.5: `post_phase()`: publish a round's tasks, then block until every one of them has been counted as done.

```
static void post_phase(mergesort_pool_t *p, unsigned ntasks)
{
    pthread_mutex_lock(&p->lock);
    p->ntasks = ntasks;
    p->next_task = 0;
    p->tasks_done = 0;
    pthread_cond_broadcast(&p->work_ready);
    while (p->tasks_done < p->ntasks)
        pthread_cond_wait(&p->phase_done, &p->lock);
    pthread_mutex_unlock(&p->lock);
}
```

The worker side is the other half of the same protocol: take the lock, wait on `work_ready` until a task index is available, copy that index out, and release the lock *before* running the task the sort itself never runs under the lock. When it finishes it takes the lock again, increments the count and the task counter, and the worker that reaches `ntasks` signals `phase_done`. The caller wakes, and only then posts the merges that read what those tasks wrote. There is no sleep anywhere, so a slow machine simply takes longer; it cannot produce a different order. The buffers and the task array are changed only between phases, which is after every task has been counted, so a worker never reads a task that is being rewritten.

3.4.4 Refusals, and the plan line

Two decisions keep the parallel path honest. Inputs below `MS_PARALLEL_MIN` (4096 values) are sorted single-threaded, because waking a pool costs more than the sort saves at that size, and `ms_sort` refuses a second concurrent call on the same pool with -1 rather than corrupting the shared round state one pool sorts one array at a time.

The plan is recorded as it happened, not as it was intended: the statistics count partitions,

merge phases and tasks actually executed. That record is what makes the parallelism checkable. "1 partition, 0 merge phases, 1 task" cannot come out of a four-worker run, so the plan line is proof that the parallel path was taken, whereas the elapsed time alone would not distinguish a fast serial sort from a parallel one.

3.4.5 Measured scaling

On the Raspberry Pi 4, sorting one 64 KiB block (voltage, thread sweep, three repeats per point):

Table 3.3: The sort alone, per 64 KiB block, against the number of workers. The plan column is what the run reported.

Workers	Plan	Time per block	Speed-up
1	1 partition, 0 phases, 1 task	2.40 ms	1.00×
2	2 partitions, 1 phase, 3 tasks	1.32 ms	1.82×
3	3 partitions, 2 phases, 5 tasks	1.09 ms	2.20×
4	4 partitions, 2 phases, 7 tasks	0.80 ms	3.00×

The separate `test_kpi` benchmark measures the same curve with a best-of-20 policy on its own block: 3.010, 1.631 and 1.011 ms at one, two and four workers, a speed-up of 2.98×, so the two measurements agree. Four workers reach three times the throughput of one rather than four: the partitions are created and the merges are contended for, and the filter that follows is serial.

The headroom is worth stating in the same breath. One block is 163.84 s of signal at the 100 Hz design rate, and it sorts in 0.80 ms roughly two hundred thousand times faster than the data arrives. Even at the recording's own, unevenly sampled rate of about 13.36 Hz, a block spans 1226 s of stream.

3.5 Rolling Median Filter (Hampel Filter)

The server implements a rolling Hampel filter to detect and suppress outliers in time-series sensor data while preserving samples that follow the local signal trend. Unlike a conventional moving-median filter that replaces every sample with the median of its neighbourhood, the Hampel filter replaces a sample only when it deviates significantly from the local median.

3.5.1 Window-Based Median and MAD Calculation

For a sample x_i, the filter extracts a sliding window of odd width w, centred on the current sample. The local median is calculated as

$$m_i = \text{median}\left(x_{i-h}, \ldots, x_i, \ldots, x_{i+h}\right), \qquad h = \frac{w-1}{2}.$$

The median absolute deviation (MAD) measures the spread of the samples around this local median:

$$\text{MAD}_i = \text{median}_{j \in W_i} \left| x_j - m_i \right|.$$

The robust estimate of the local standard deviation is obtained using the scaling factor 1.4826:

$$\sigma_i^* = 1.4826 \ \text{MAD}_i \, .$$

A sample is classified as an outlier if

$$|x_i - m_i| > \lambda \sigma_i^*,$$

where λ is the configurable threshold. The implementation uses a default threshold of 4.0. When the condition is satisfied, the output sample is replaced by the local median; otherwise, the original sample is retained.

3.5.2 Filter Configuration and Boundary Handling

The filter supports odd window widths from 3 to 31 samples, with a default width of 9. A width of 9 considers up to four neighbouring samples on each side of the current sample. A larger window can detect wider bursts of abnormal samples, but it increases computational cost and may smooth legitimate short-duration variations.

The filter processes samples in their original time order. Since sorting destroys temporal order, the server maintains separate data paths for sorting and filtering rather than applying the rolling filter to the sorted array.

For block-based processing, neighbouring samples from adjacent blocks are supplied as context. This prevents artificial discontinuities at block boundaries and makes block-wise

filtering equivalent to filtering the complete signal, provided the required context is supplied. At the beginning and end of the complete recording, the window is clipped to the samples that actually exist.

The implementation is deterministic and does not allocate memory dynamically during filtering. Each local window is ordered using insertion sort, allowing the median and MAD to be calculated without a separate general-purpose sorting operation.

3.6 Client Implementation

The client acts as the orchestrator of the pipeline. Its responsibilities include reading the recording, preparing 64 KiB blocks, submitting them to the server, and retrieving the results. Because the system relies on asynchronous events, the client must set up its own receiving channel using `name_attach()` before it even connects to the server. If the server is not yet running, the client waits rather than failing immediately.

3.6.1 Connection Deadline

A strict requirement of the project is that the client must tolerate the server starting slightly late. The client implements a 10-second deadline by repeatedly calling `name_open()` with a 250 ms `nanosleep` delay between attempts. If the server appears within this window, the connection succeeds and the wait time is logged. If the deadline expires, the client prints a descriptive error and exits cleanly, avoiding an indefinite hang.

Listing 3.6: Retrying the server connection with a 10-second deadline

```
1  static int connect_to_server(const char *name, uint64_t deadline_ns,
2                               uint64_t *waited_ns, int *attempts)
3  {
4      uint64_t start = now_ns();
5      int tries = 0;
6
7      for (;;) {
8          struct timespec pause;
9          uint64_t elapsed;
10         int coid = name_open(name, 0);
11
```

```
12          tries++;
13          if (coid != -1) {
14              if (waited_ns != NULL) *waited_ns = now_ns() - start;
15              if (attempts != NULL) *attempts = tries;
16              return coid;
17          }
18          elapsed = now_ns() - start;
19          if (elapsed >= deadline_ns) {
20              if (waited_ns != NULL) *waited_ns = elapsed;
21              if (attempts != NULL) *attempts = tries;
22              return -1; /* errno is the last name_open failure */
23          }
24          pause.tv_sec = 0;
25          pause.tv_nsec = (long)CONNECT_RETRY_NS;
26          (void)nanosleep(&pause, NULL);
27      }
28  }
```

3.6.2 Pipeline Orchestration and Block Preparation

The client drives the local data pipeline before interacting with QNX. Before the transmission loop begins, it calls `block_loader_open()` to initialize the virtual sensor and prepare the file layout. Inside the loop, it calls `block_loader_next()` to read the 16,384 samples from disk, zero-pad the final block if necessary, and generate the block descriptor. Only after this local preparation is complete does the client package the header using `proto_submit_prepare()` and trigger the IPC mechanisms. The complete execution flow is shown in Figure 3.4.

3.6.3 Two-Part IOV Submission

To transmit a 64 KiB block, the client avoids copying the payload into a single, massive message buffer. Instead, it uses an Input/Output Vector (IOV) to send the message in two separate parts: the small `proto_submit_t` header and the 64 KiB `proto_block_t` payload.

The client arms a reply deadline using `TimerTimeout` and calls `MsgSendv()`. This allows the QNX kernel to transfer the header and data directly from their respective memory

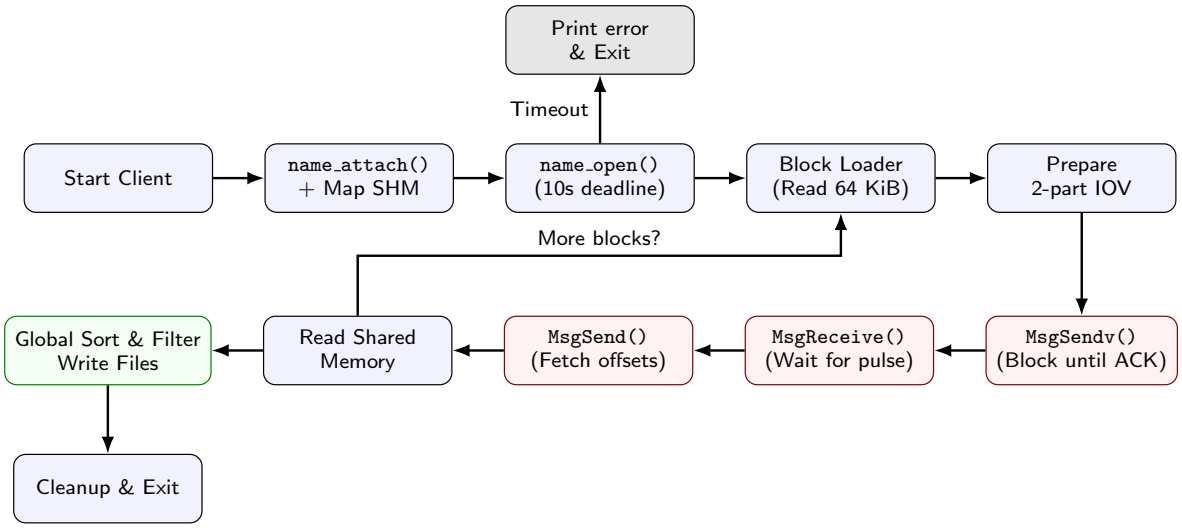


Figure 3.4: Execution flow of the Client. The client loops through the blocks, utilizing QNX message passing. Once the recording ends (EOF), it performs a global sort, filters the data, writes the output files, and exits.

locations. The client then blocks until it receives the server's acknowledgement (ACK), verifying that the message type, job ID, and checksum status are all correct.

Listing 3.7: Submitting a block using a two-part IOV

```
1       iov_t send_iov[2], reply_iov;
2
3       /* ... prepare submit and block ... */
4
5       SETIOV(&send_iov[0], &submit, sizeof(submit));
6       SETIOV(&send_iov[1], &block, sizeof(block));
7       SETIOV(&reply_iov, &ack, sizeof(ack));
8
9       arm_reply_deadline("submit");
10
11      if (MsgSendv(server_coid, send_iov, 2, &reply_iov, 1) == -1) {
12          perror("client: MsgSendv submit");
13          goto cleanup;
14      }
15
16      if (ack.type != PROTO_MSG_ACK || ack.job_id != job_id ||
17          ack.status != PROTO_OK) {
```

```
18            fprintf(stderr, "client: submit %u rejected: %s\n",
19                    job_id, proto_status_str(ack.status));
20            goto cleanup;
21        }
```

3.6.4 Completion Pulse Handling

Because the server processes the block asynchronously, it sends a QNX pulse when the work is done. The client waits for this pulse using `MsgReceive()` with a 5-second timeout. A QNX pulse can only carry a 32-bit value; however, an IEEE 754 float is also exactly 32 bits. The server bit-casts the calculated median directly into the pulse value. The client extracts this 32-bit integer and bit-casts it back into a float, receiving the median instantly without needing a separate message round-trip.

3.6.5 Shared Memory Retrieval

After receiving the pulse, the client sends a fetch message to get the result descriptor, which contains the exact byte offsets of the sorted and filtered arrays.

Instead of transferring 128 KiB (two 64 KiB arrays) back through message passing, the client maps the server's named shared memory object as read-only. It validates the header and the job ID to ensure the slot hasn't been overwritten, and then reads the arrays directly using the supplied offsets. Finally, it recalculates the FNV-1a checksums of the arrays in memory to prove that the data was not corrupted in transit.

3.6.6 Final Verification and Output

Once all blocks have been offloaded (the loader returns 0), the client's work is not yet done. It performs a local verification and post-processing phase:

- It sorts the entire recording locally using a serial merge-sort to verify that its global median matches the median the server sent in the final pulse.
- It runs a continuous rolling median filter(Hampel) over the entire raw recording to ensure that block boundaries do not cause artifacts in the final output.
- It writes the processed arrays to disk as `raw.bin`, `sorted.bin`, and `filtered.bin`, alongside a `medians.csv` file containing the per-block statistics.

3.7 Server Implementation

The server handles the processing of incoming data blocks. It runs as a QNX channel server, where it receives client requests, checks their validity, and assigns the processing tasks to a pool of worker threads. This allows the server to accept new requests without waiting for the sorting and filtering of previous blocks to finish.

3.7.1 Initialization and Resource Allocation

Before starting the receive loop, the server initializes the resources required for processing. It calls `shm_region_create()` to create and map a named shared memory region that stores the sorted and filtered arrays. Next, `ms_pool_create()` creates the worker thread pool used for parallel merge sort, with the default configuration using one thread per CPU core. Finally, `name_attach("sortmed")` registers the server under its service name and creates the QNX channel through which it receives client requests.

3.7.2 Two-Stage Reception and Validation

Since each data block is 64 KiB, the server checks the request header before accepting the entire payload for processing. It first uses `MsgReceivev()` to receive the 48-byte `proto_submit_t` header. The server checks the protocol version and validates the filter width using `proto_submit_valid()`. It then calls `proto_submit_verify()` to verify the block checksum.

If any validation step fails, the server returns an appropriate error status using `MsgReply()` and skips further processing of that request. This helps avoid unnecessary work on invalid input.

Listing 3.8: Receiving and validating the request header

```
1       rcvid = MsgReceivev(attach->chid, riov, 2, &info);
2       /* ... */
3       if (type == PROTO_MSG_SUBMIT) {
4           proto_ack_t ack;
5           memset(&ack, 0, sizeof(ack));
6           ack.type = PROTO_MSG_ACK;
7           ack.job_id = submit.job_id;
8
9           if (submit.version != PROTO_VERSION) {
```

```
10              ack.status = PROTO_ERR_VERSION;
11              (void)MsgReply(rcvid, 0, &ack, sizeof(ack));
12              continue;
13          }
14          if (!proto_submit_valid(&submit)) {
15              ack.status = PROTO_ERR_ARGUMENT;
16              (void)MsgReply(rcvid, 0, &ack, sizeof(ack));
17              continue;
18          }
19          if (!proto_submit_verify(&submit, &block)) {
20              ack.status = PROTO_ERR_CHECKSUM;
21              (void)MsgReply(rcvid, 0, &ack, sizeof(ack));
22              continue;
23          }
```

3.7.3 Early Acknowledgement

The server does not keep the client waiting until all the processing is complete. In QNX, the `rcvid` identifies a received message and is used to reply to that request. Once the reply is sent, that receive ID can no longer be used for the same request.

After validating the submission, the server therefore sends an acknowledgement (ACK) before the computational work begins. The client can then continue with its workflow and wait for a separate completion pulse, while the server proceeds with processing the submitted block.

3.7.4 Block Processing and Function Calls

After acknowledging the request, the server processes the data block through `process_submit()`. The function copies the input into a separate array so that the original time order is preserved for filtering. It then performs the following operations:

1. Copies the input samples into a separate array using `memcpy()`, preserving the original data order.

2. Calls `ms_sort()` to sort the copied samples using the parallel worker pool.

3. Calls `median_filter_block()` to apply the Hampel filter to the original time-ordered samples.

4. Uses `proto_checksum()` to calculate the FNV-1a checksums of the sorted and filtered arrays.

5. Stores the result metadata, including the median, filter width, number of outliers, and checksums, in the `proto_result_t` structure.

Listing 3.9: Sorting, filtering, and preparing the block results

```
1  /* Preserve the original sample order for filtering. */
2  memcpy(sorted, block->samples, n * sizeof(float));
3
4  /* Sort the block using the parallel worker pool */
5  if (ms_sort(pool, sorted, n) != 0) return -1;
6
7  /* Apply the Hampel filter to the original time-ordered block */
8  if (median_filter_block(&cfg, NULL, 0, block->samples, n, NULL, 0,
9                          filtered, &outliers) != 0)
10     return -1;
11
12 /* Store the result metadata and checksums */
13 job->result.median = median_of_sorted(sorted, n);
14 job->result.filter_width = cfg.width;
15 job->result.outliers_removed = outliers;
16 job->result.sorted_checksum = proto_checksum(sorted, n * sizeof(float));
17 job->result.filtered_checksum = proto_checksum(filtered, n * sizeof(float))
       ;
```

3.7.5 Global Run Aggregation

In addition to processing individual blocks, the server calculates the median of the complete recording. To do this, it maintains a global sorted array that is updated as each block is processed. The function `run_merge_block()` merges each block's sorted samples into this array. When

a submission carries the PROTO_SUBMIT_LAST flag, the server calls run_publish() to store the final proto_run_t descriptor in shared memory, including the global median.

3.7.6 Asynchronous Pulse Notification

After processing a block, the server notifies the client that the results are ready. Since the original request has already been acknowledged, its rcvid is no longer available for notification. The server instead uses name_open() to obtain a connection to the client's registered channel, sortmed-client.

The server converts the block median into a 32-bit pulse value using proto_float_to_pulse(). This allows the median to be included in the completion pulse, along with a code derived from the job ID. The server sends the pulse using MsgSendPulse(). Once the notification attempt is complete, it closes the connection using name_close().

Listing 3.10: Sending a completion pulse to the client

```
1           float pulse_median = jobs[free_index].result.median;
2           int coid = name_open(CLIENT_NAME, 0);
3
4           if (coid == -1 ||
5               MsgSendPulse(coid, SIGEV_PULSE_PRIO_INHERIT,
6                            PROTO_PULSE_DONE + (int)submit.job_id,
7                            proto_float_to_pulse(pulse_median)) == -1) {
8               perror("sortmed-server: completion pulse");
9           }
10          if (coid != -1) {
11              name_close(coid);
12          }
```

3.8 QNX IPC Mechanisms

The system uses three QNX inter-process communication (IPC) mechanisms to exchange requests, notify the client, and transfer the processed results. Each mechanism serves a different purpose in the processing pipeline.

- **Message Passing (IOV):** The client uses MsgSendv() to send the 48-byte header and the

64 KiB data block using separate I/O vector entries in a single call. This allows the header and payload to be provided from different memory locations without first combining them into a single buffer.

- **QNX Pulses:** Since processing takes several milliseconds, the server sends an acknowledgement (ACK) after validating the request instead of keeping the client waiting for the entire operation. Once processing is complete, it uses `MsgSendPulse()` to notify the client. The calculated median is encoded in the 32-bit pulse value field.

- **Shared Memory:** Sending the sorted and filtered arrays back through message passing would add unnecessary data transfer overhead. The server instead writes both arrays to the named shared memory object `/dev/shmem/sortmed-results`. The client maps this region as read-only and accesses the results directly, avoiding the need for another large message transfer.

3.9 End-to-End Pipeline Execution

The complete pipeline combines the client, server, worker threads, and shared memory to process each 64 KiB data block. The client submits the data and receives an acknowledgement before the server begins the main computation. Once processing is complete, the server notifies the client using a QNX pulse. This allows the two sides to handle submission and processing without keeping the original request open until the results are ready.

Figure 3.5 shows the overall data flow for a single block.

3.9.1 Phase 1: Local Preparation and Submission

The process starts on the client side, where `block_loader_next()` reads up to 16,384 samples from the dataset. If the final block contains fewer samples, the remaining positions are zero-padded. The client then calculates the FNV-1a checksum and prepares the request using a 48-byte `proto_submit_t` header and a 64 KiB `proto_block_t` payload.

The client submits both parts using `MsgSendv()`. On the server side, `MsgReceivev()` receives the request, after which the server checks the protocol version, filter width, and checksum. If the request passes validation, the server sends an acknowledgement (ACK) to the client.

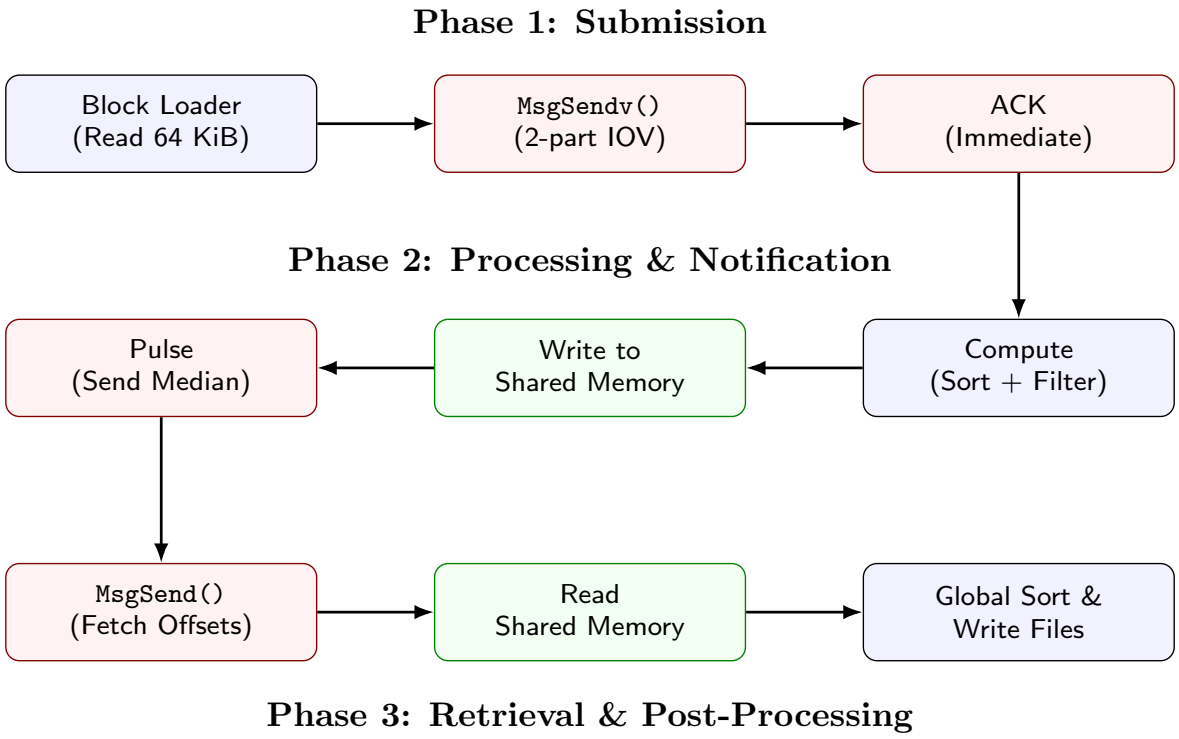


Figure 3.5: End-to-end execution of a 64 KiB data block. Message passing handles requests and control signals, shared memory stores the processed arrays, and QNX pulses notify the client when processing is complete.

The ACK is sent before the main processing begins. This allows the client to continue its execution and wait for a completion pulse rather than remaining blocked until sorting and filtering are finished. The original request is therefore completed separately from the later notification of the results.

3.9.2 Phase 2: Computation and Publication

After acknowledging the request, the server processes the submitted block using `process_submit()`. The main operations are:

1. The raw samples are copied into a designated shared memory slot, preserving their original time order.
2. The server calls `ms_sort()` to sort the samples using the worker thread pool. The sorted data is stored separately from the raw samples.
3. The function `median_filter_block()` applies the Hampel filter to the original time-ordered data and stores the filtered output in another slot.

4. The server calculates the output checksums and fills the `proto_result_t` descriptor with the result metadata.

After preparing the results, the server obtains a connection to the client's registered channel and sends a completion pulse using `MsgSendPulse()`. The calculated median is encoded in the 32-bit pulse value, while the pulse code identifies the job. The client can therefore receive the completion notification without requiring the server to reopen the original request.

3.9.3 Phase 3: Retrieval and Post-Processing

The client waits for the completion pulse using `MsgReceive()`. When the pulse arrives, it extracts the median and sends a separate fetch request using `MsgSend()` to obtain the result descriptor. This descriptor provides the offsets of the sorted and filtered arrays in shared memory.

Since the client has already mapped the shared memory region as read-only, it can access the arrays directly using these offsets. It also recalculates their FNV-1a checksums to verify that the retrieved data matches the expected results.

After all blocks have been processed, the client completes the remaining post-processing steps. These include calculating the global median, applying the required final filtering across the recording, and writing the output files: `raw.bin`, `sorted.bin`, `filtered.bin`, and `medians.csv`.

3.10 Testing

Every stage of this service has a plausible-looking failure mode. A sort that loses one value, a filter that replaces a legitimate edge, a checksum computed over the wrong bytes, a shared region read at the wrong offset: each still produces numbers of the right shape, and inspection of the output alone would not settle any of them. The tests are therefore built to compare against an answer computed some other way, and to make the system prove that it refuses what it should refuse.

Three words are used throughout, so they are worth defining. A *check* is one assertion: this value equals that one, this call returns -1. A *test* is a named group of checks exercising one behaviour, and a *suite* is one binary that runs a group of tests and prints a verdict. The counting

is per check because a test that quietly covers half of its cases still passes. “0 skipped” means every check that could run did run: the payload tests are skipped when the generated dataset is absent, so a run quoting zero skipped was made against the real recording.

3.10.1 Two folds, one set of sources

The tests run twice, on two very different machines, from exactly the same source files.

The *host* fold is a set of plain C programs built with `gcc`. There is no test framework: each file in `RT-MOMS/tests/` includes `test_util.h`, calls check macros and returns its own exit status, and `make check` builds and runs the six suites in order. It is fast, it can be instrumented, and it can do the two things the target cannot do comfortably: compare against the C library’s `qsort`, and feed malformed input on purpose.

The *board* fold cross-compiles the same sources with `qcc -Vgcc_ntoaarch64le` and runs them from the boot image on the Raspberry Pi 4, where the compiler, the C library and the CPU are all different. The board returns the same counts for the same suites, which is the point: with the code held constant, the only thing left that could differ is the kernel, and that is what the message passing, the pulse and the shared results region exercise. The board run is therefore a confirmation of the host run rather than a separate experiment; a disagreement would point at the environment, and agreement means the behaviour is not an accident of one toolchain.

3.10.2 Four kinds of evidence

Table 3.4: The kinds of evidence the suites use, and why each is convincing.

Kind	Example	Why it is convincing
Two independent routes	planar file versus interleaved record; one sample at a time versus whole blocks	different code paths, so a stride, offset or window-refill mistake makes them disagree
| An oracle	`qsort` on the same values; a reference Hampel implementation for the filter	not our code and not our bug
Invariants	output ordered; the multiset unchanged; seals match; byte-identical output for 1, 2, 3, 5 and 8 workers	catches a class of bug rather than one example: a lost value, a duplicated value, a half-merged phase
Deliberate bad input	a 3-byte file, a 6-float record, a NULL path, one nudged sample, a fetch for an unknown job	proves the system refuses instead of quietly doing something wrong

3.10.3 The suites and what they establish

Table 3.5: The six suites. The checks are per suite; the total is the figure quoted for both folds. The KPI suite is a benchmark and is counted separately.

Suite	Checks	What it establishes
`test_protocol`	186	the wire contract: 64 KiB blocks, safe message numbers, seal sensitivity, shared-region arithmetic
`test_sensor`	642	a file behaves like a device: both layouts agree bit for bit, frames capped, clock exact, junk refused
`test_block_loader`	116	a stream becomes blocks that survive the trip: split right, padded right, sealed over the right bytes
`test_mergesort`	578	sorted, the same multiset, byte-identical for 1, 2, 3, 5 and 8 workers, plan line as expected
`test_median`	7 615	the Hampel filter against a reference, across block seams, bursts, dropouts and bad arguments
`test_shared_memory`	13	the results region's bounds and arithmetic
`test_kpi`	79	benchmark: sort at 1 / 2 / 4 workers, global median computed two ways
Total	**9 150**	0 failed, 0 skipped, on host and on board

3.10.4 The board run

Listing 3.11: The board's summary lines, assembled here with the suite names added; the screenshots that follow are the console itself.

```
Shared-memory bounds tests: 13 checks, 0 failures

  186 checks, 0 failed, 0 skipped (protocol)
  7615 checks, 0 failed, 0 skipped (median)
  578 checks, 0 failed, 0 skipped (mergesort)
  642 checks, 0 failed, 0 skipped (sensor)
  116 checks, 0 failed, 0 skipped (block loader)
  79 checks, 0 failures (kpi benchmark)

client-wait-test: both cases pass
soak: 20 runs, all completed, all with identical medians
```

The screenshots that follow are the console as it was captured, one per suite, plus the two board scripts, `client-wait-test.sh` and `soak.sh`. They are the evidence rather than the summary: the test names scroll past with their verdicts, and the suites that reject bad input print the refusals they provoked; every one of those lines is followed by `ok`.

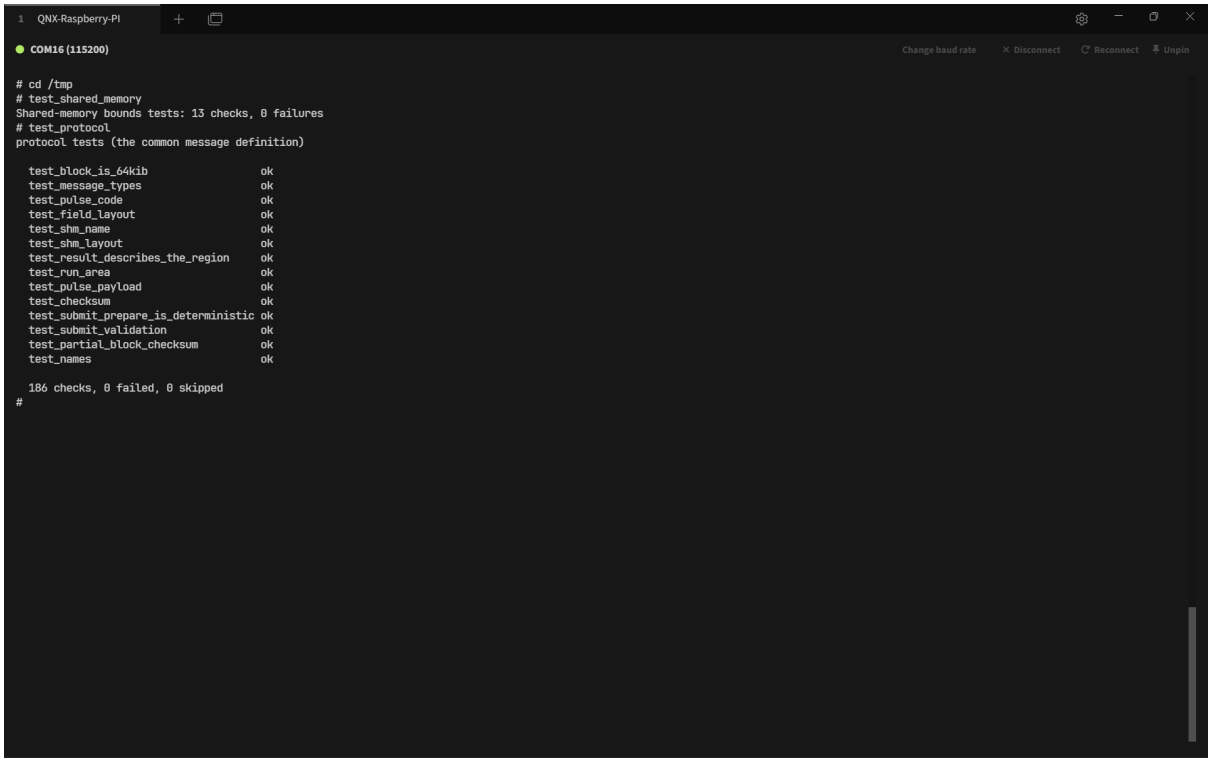


Figure 3.6: The protocol suite on the board: the wire contract, the shared-region layout and the seal's sensitivity, 186 checks.

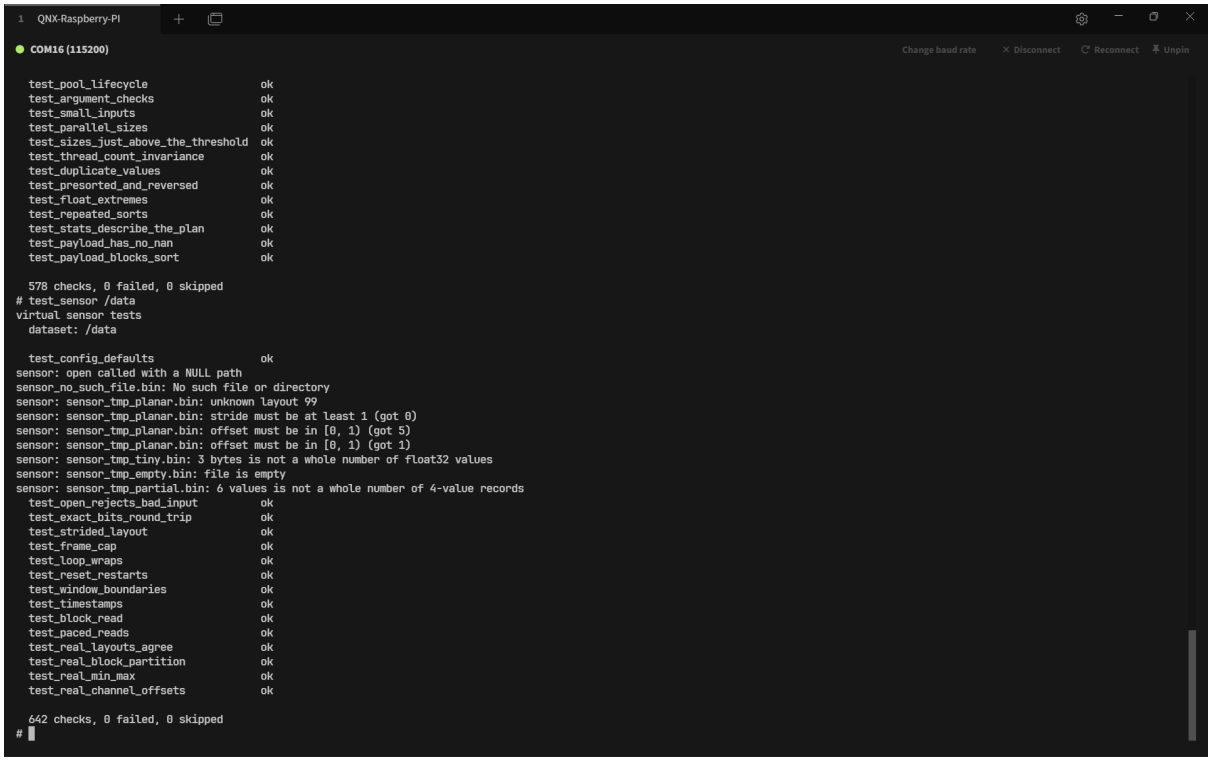


Figure 3.7: The sensor suite on the board. The `sensor:` lines are the test failing on purpose (a missing file, a 3-byte file, a 6-float record), each followed by `ok`.

One limitation these runs do not settle. The client's wait for the completion pulse has no timeout, so a server that died mid-job would leave the client waiting; the deadline test exercises the deadline on connection, not on completion.

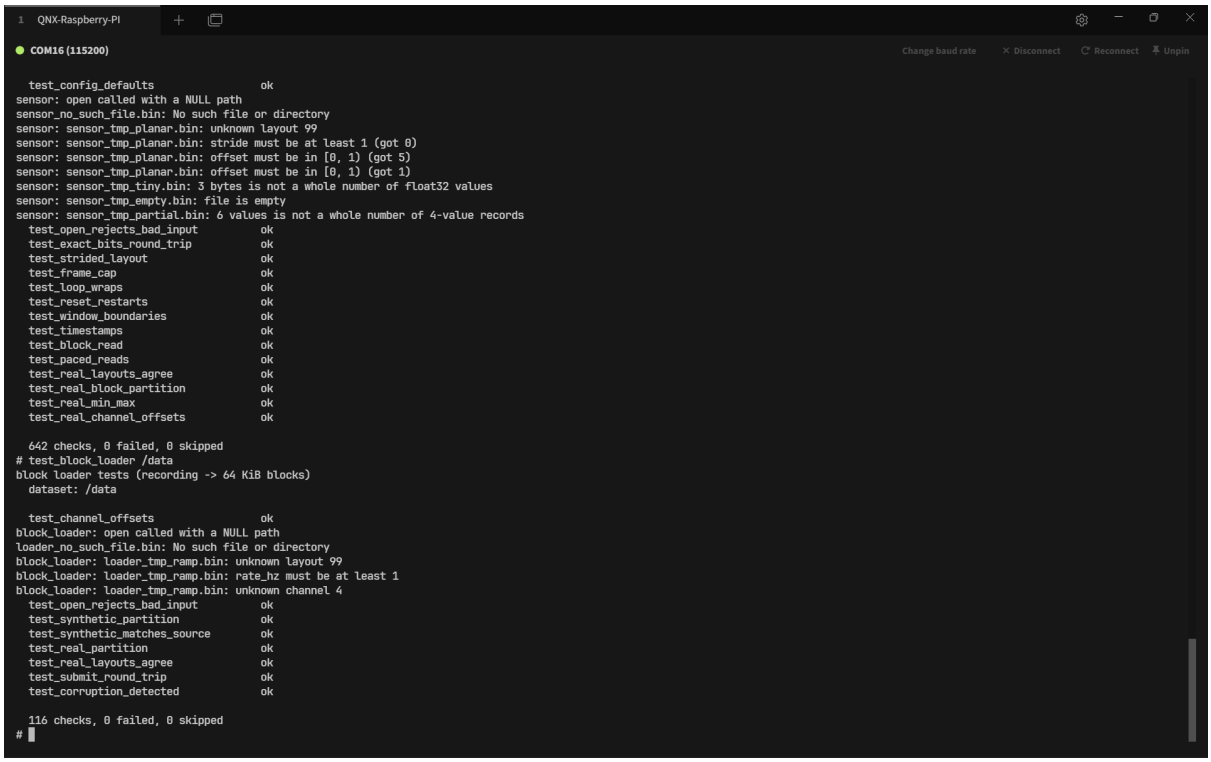


Figure 3.8: The block loader suite on the board: the interleaved recording produces the same blocks and the same seals as the host.



Figure 3.9: The merge sort suite on the board: ordering, the multiset, the plan line and byte-identical output across thread counts.

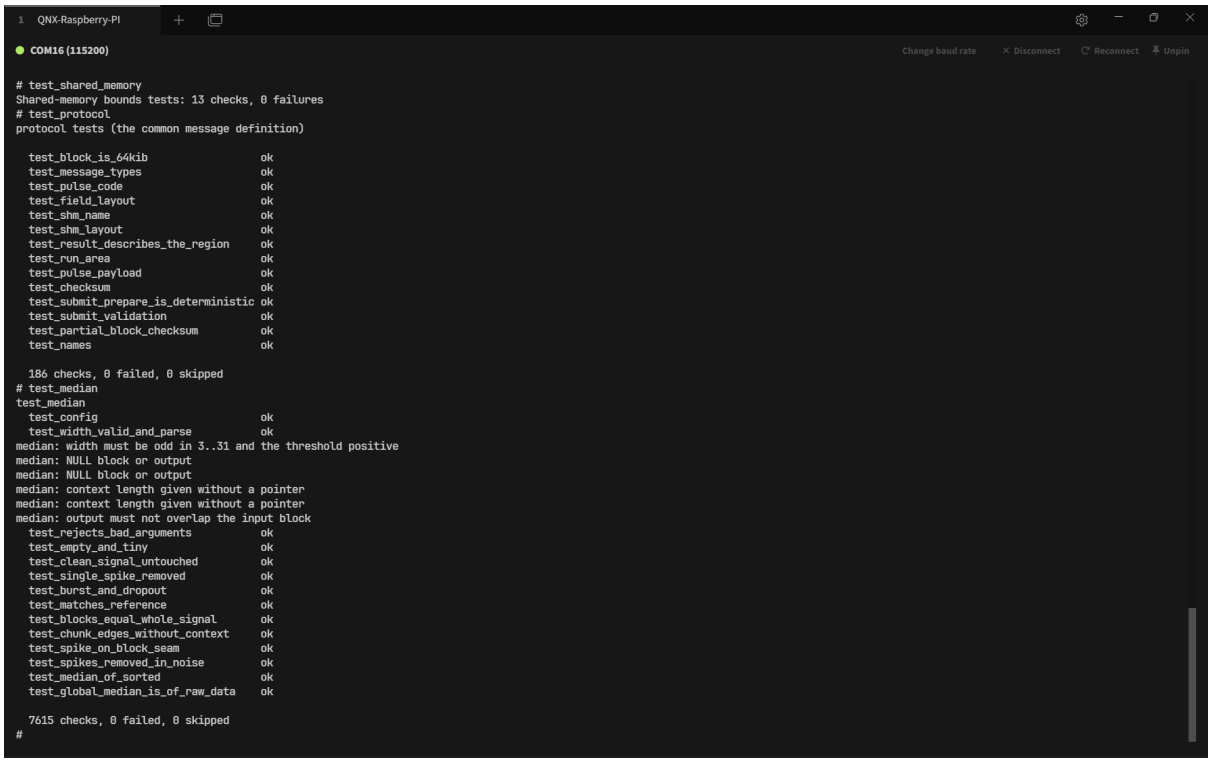


Figure 3.10: The median suite on the board, the largest at 7 615 checks: the filter against a reference, across seams, bursts and dropouts.

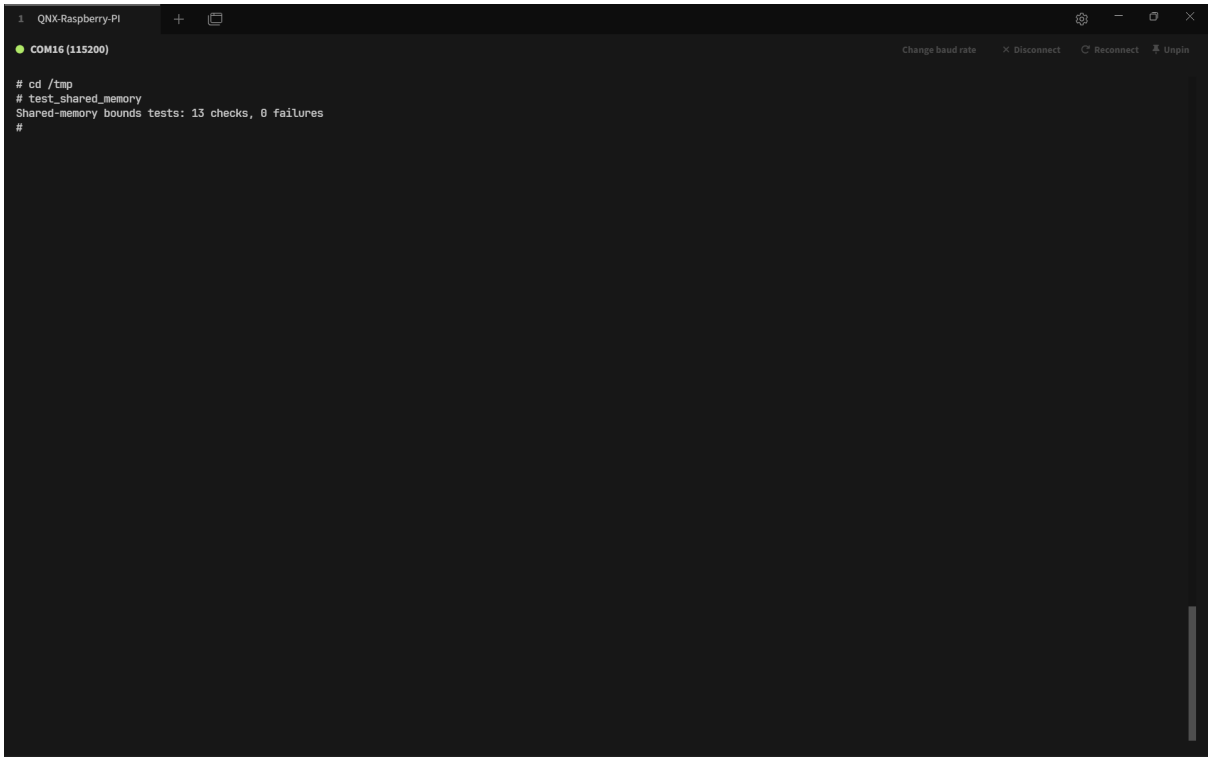


Figure 3.11: The shared-memory suite: the results region's bounds and arithmetic, 13 checks.

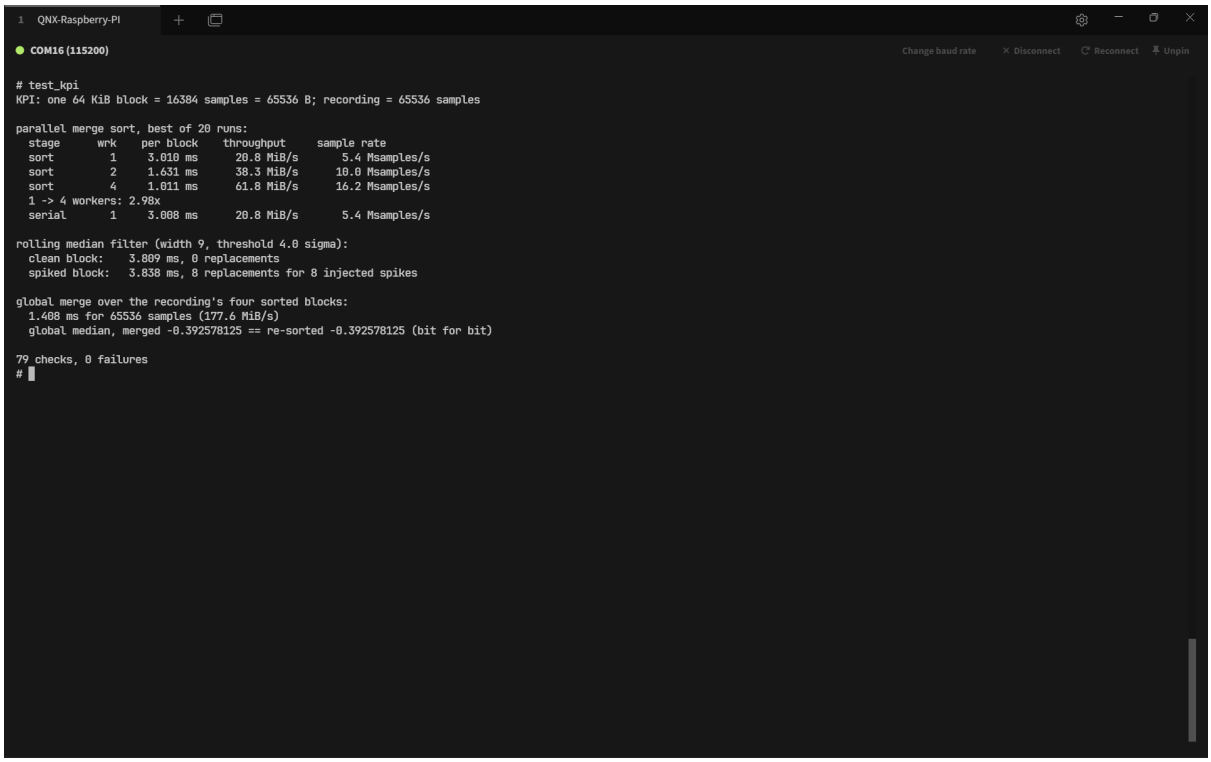


Figure 3.12: The KPI benchmark on the board: sort throughput at one, two and four workers, and the global median computed two ways agreeing bit for bit.

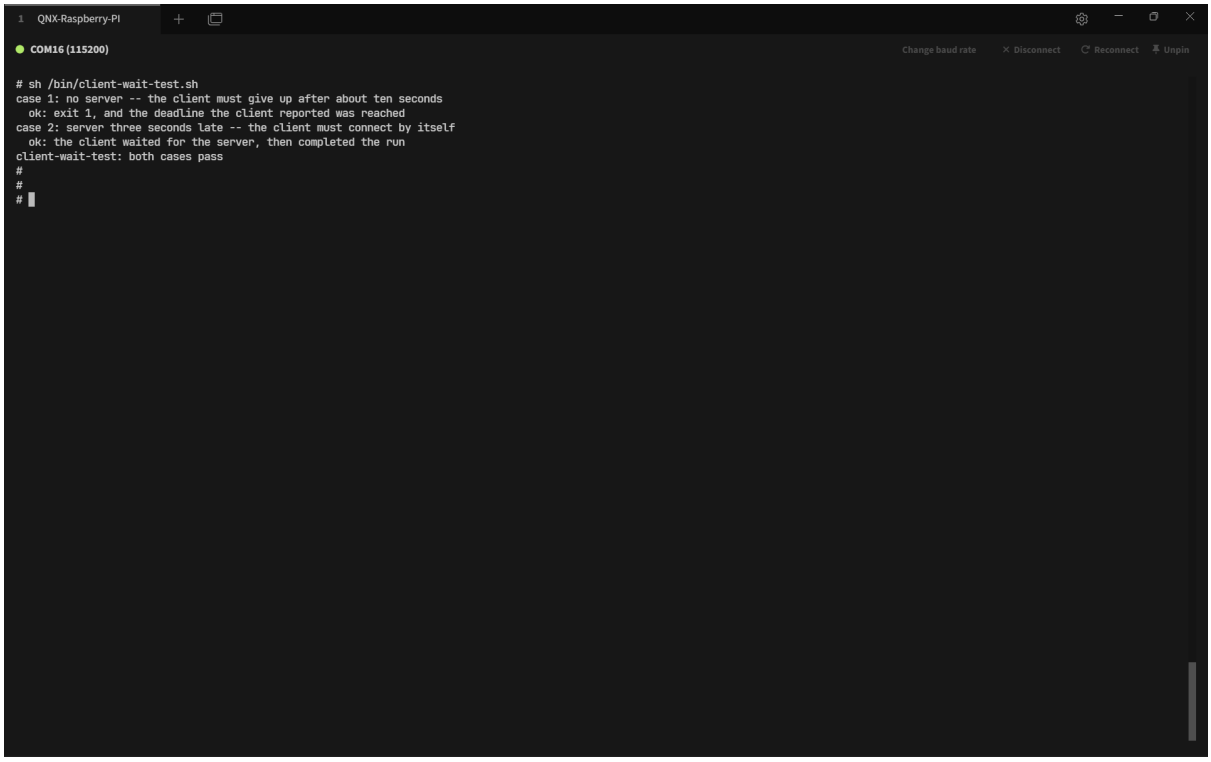


Figure 3.13: The client deadline test: with no server the client gives up at ten seconds, and with a server that starts three seconds late it connects by itself.



Figure 3.14: The soak: twenty consecutive runs, every one completing with identical medians and a stable service time per block.

Chapter 4

Results and Discussions

4.1 Experimental Setup

The experiments were conducted on a Raspberry Pi 4 running QNX Neutrino RTOS. The target system was used to evaluate the complete client-server pipeline, including block transfer, parallel merge-sort, rolling Hampel filtering, shared-memory result retrieval and completion notification.

The input was a recorded battery sensor dataset, `Chen2020_c1_T25_discharge.bin`, replayed through the virtual sensor interface. The recording contains 60,000 samples per selected channel, stored as single-precision floating-point values. The configured nominal sensor rate was 100 Hz, and the recording used the interleaved layout containing time, voltage, current and temperature.

The client assembled blocks of 16,384 samples, corresponding to 64 KiB per full block. A complete 60,000-sample recording was processed in four blocks, with the last block containing the remaining 10,848 samples.

The server used a four-worker merge-sort pool for the window-width experiments. The filter-width sweep covered widths of 3, 5, 7, 9, 11, 13, 15, 21 and 31 samples, with three repetitions per channel and width. A separate worker-count sweep evaluated one, two, three and four workers at the default filter width of 9.

The evaluation included functional correctness, filter quality, processing time, repeated-run consistency and shared-memory bounds. Since the input was replayed from a recording, the experiments validate execution on the QNX hardware target but do not represent measurements from a physical sensor connected directly to the board.

4.2 Filter Quality and Outlier Removal (Width Sweep)

4.2.1 Overall Filter Quality

The effect of filter width was evaluated on the Raspberry Pi 4 running QNX Neutrino RTOS. The experiments used the recorded battery sensor dataset and tested window widths of 3, 5, 7,

9, 11, 13, 15, 21 and 31 samples. Each configuration was executed three times.

Filter quality was evaluated for the voltage, current and temperature channels using outlier-detection recall, the rate of replacing samples not marked as outliers, and the change in reconstruction error after filtering. These metrics assess both the ability to suppress abnormal readings and the risk of altering valid measurements.

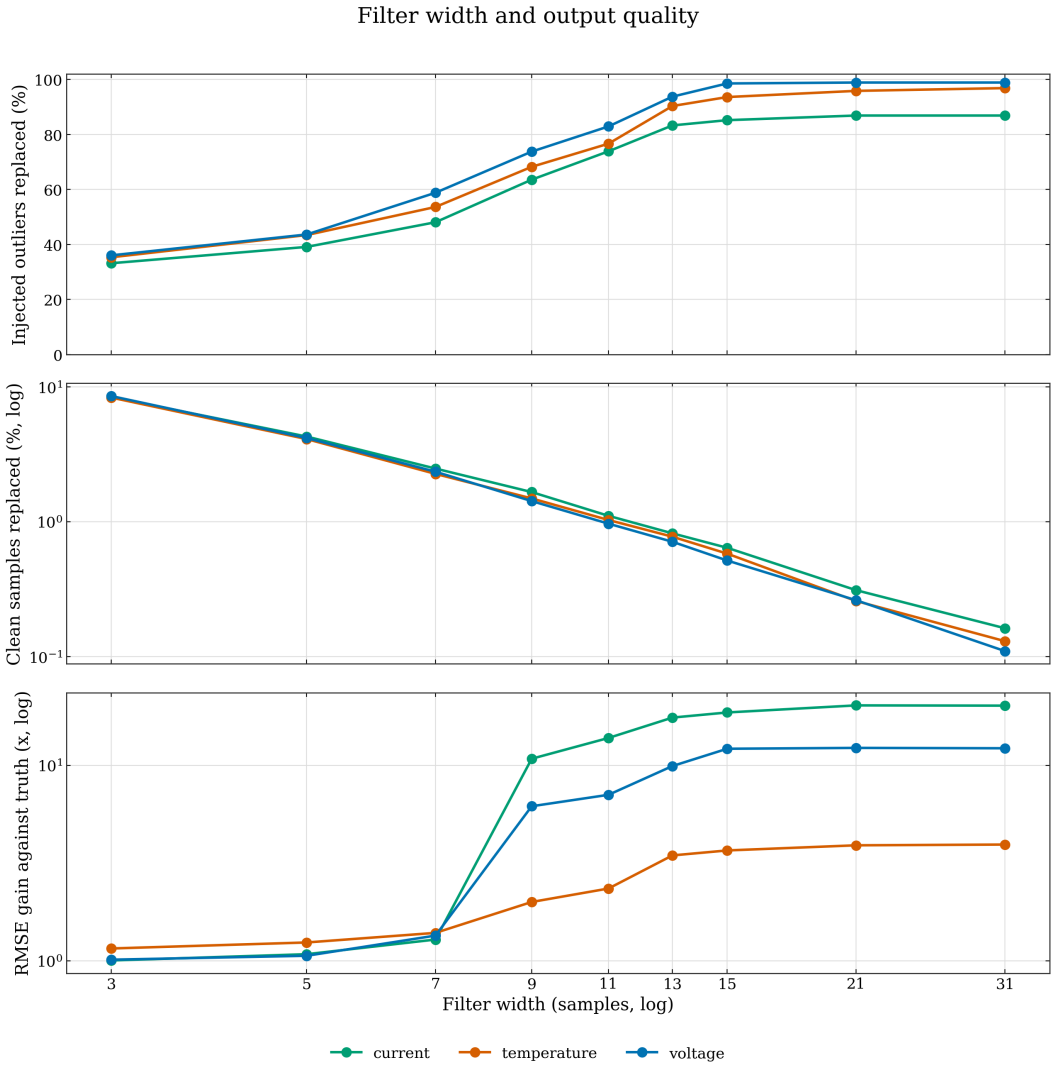


Figure 4.1: Filter quality metrics for different rolling-window widths.

The width sweep illustrates the trade-off between outlier detection and preservation of clean samples. A wider window provides a larger neighbourhood for estimating the local median and MAD. However, increasing the window width also increases computational cost and may affect legitimate short-duration signal variations.

The error map complements the quality metrics by showing the distribution of errors across the evaluated data. The quality and error plots should be interpreted together, since reducing reconstruction error does not necessarily mean that every injected outlier has been



Figure 4.2: Error map comparing the raw and filtered sensor data against the reference signal.

detected or that every valid sample has been preserved.

The numerical results also demonstrate the effect of window width. When the width increases from 9 to 15 samples, voltage outlier recall increases from 73.77% to 98.55%, current recall increases from 63.57% to 85.20%, and temperature recall increases from 68.24% to 93.60%. The corresponding RMSE decreases from 0.0039 to 0.0020 for voltage, from 0.0147 to 0.0085 for current, and from 0.1170 to 0.0638 for temperature.

These results indicate that a wider window performs better for the injected outlier patterns in this dataset. However, the additional processing cost must also be considered when selecting the operating width. The default width of 9 is therefore a configuration choice rather than a universally optimal setting.

Table 4.1: Filter quality comparison for window widths of 9 and 15 samples.

Channel	Recall (9)	Recall (15)	RMSE (9)	RMSE (15)
Voltage	73.77%	98.55%	0.0039	0.0020
Current	63.57%	85.20%	0.0147	0.0085
Temperature	68.24%	93.60%	0.1170	0.0638

Table 4.1 shows that increasing the window width from 9 to 15 samples improves outlier recall and reduces RMSE across all three evaluated channels. This indicates improved detection of the injected outliers for the tested dataset, although the increased computational cost must also be considered.

4.3 Channel-Wise Filtering Results

The effect of the rolling Hampel filter was examined separately for the voltage, current and temperature channels. For each channel, the full signal overlay shows the overall behaviour of the filter, the zoomed view highlights its local effect, and the error plot provides a complementary view of the changes introduced by filtering.

4.3.1 Voltage Channel

The voltage overlay illustrates the overall effect of filtering, while the zoomed view provides a closer examination of local variations. The error plot complements these visual observations by showing the deviation introduced by filtering. These figures should be interpreted alongside the voltage-channel recall and RMSE measurements to assess outlier suppression and signal preservation.

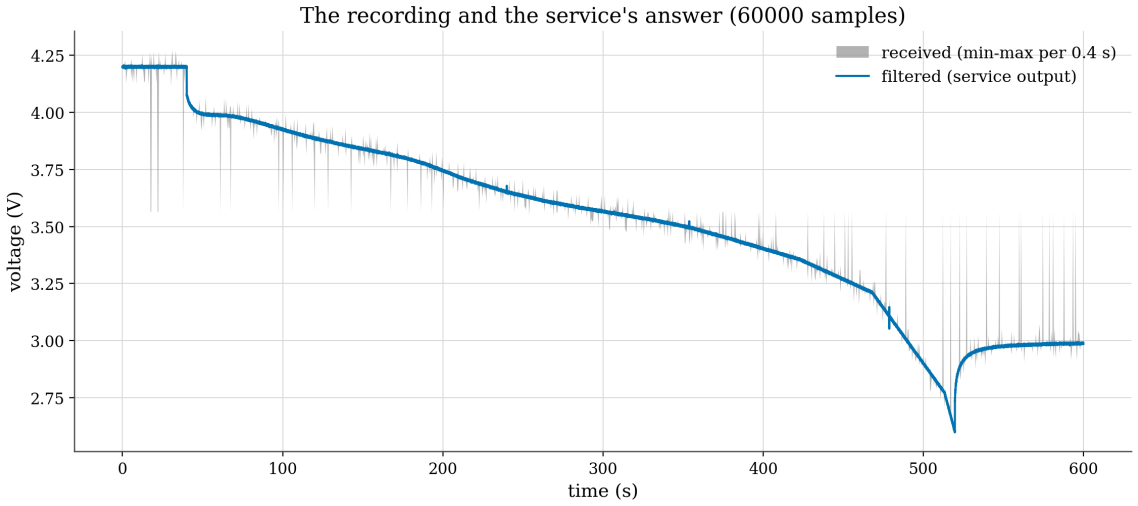


Figure 4.3: Raw and filtered voltage signals using the rolling Hampel filter.

4.3.2 Current Channel

The current-channel results show the effect of filtering on the current measurements at both the overall and local scales. The zoomed view helps examine individual signal variations, while the error plot shows the changes introduced by the filter. The results can be assessed quantitatively using the current-channel recall and RMSE values reported in the width-sweep analysis.

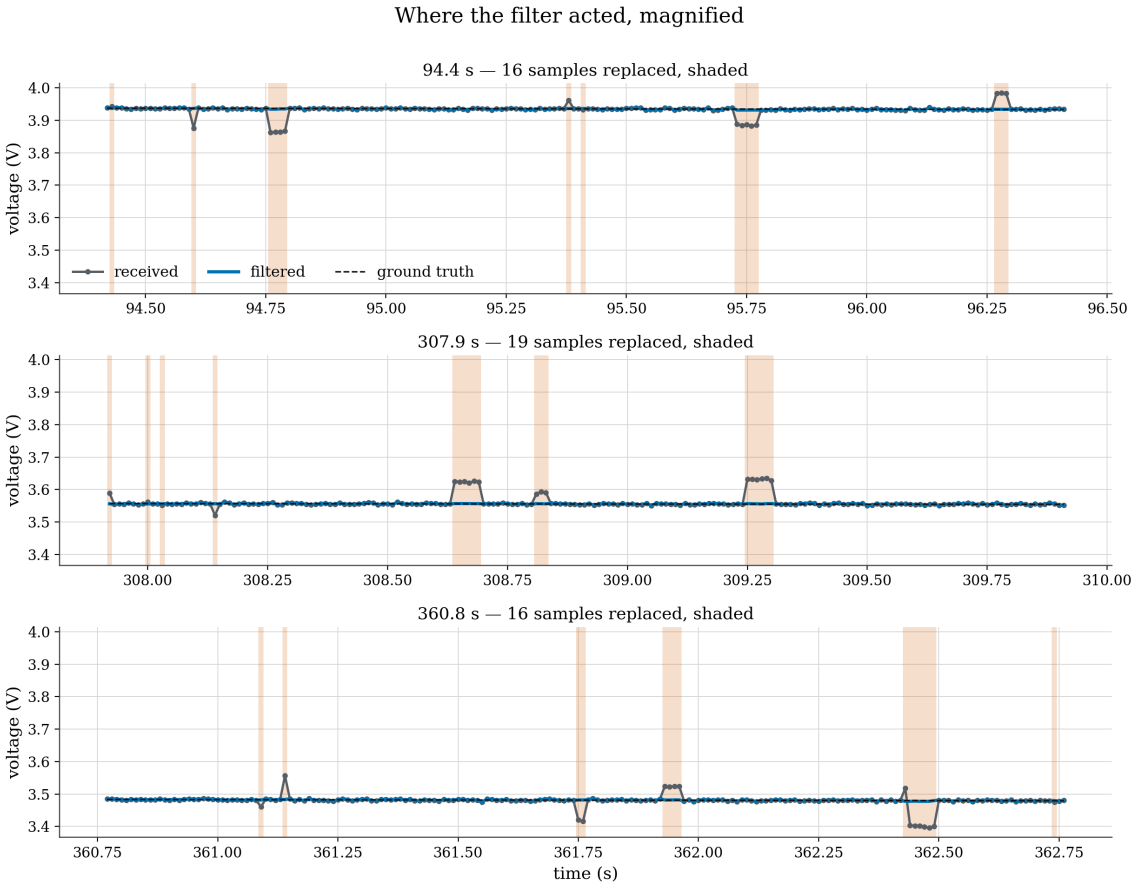


Figure 4.4: Zoomed view of the raw and filtered voltage signals, highlighting the local effect of Hampel filtering.

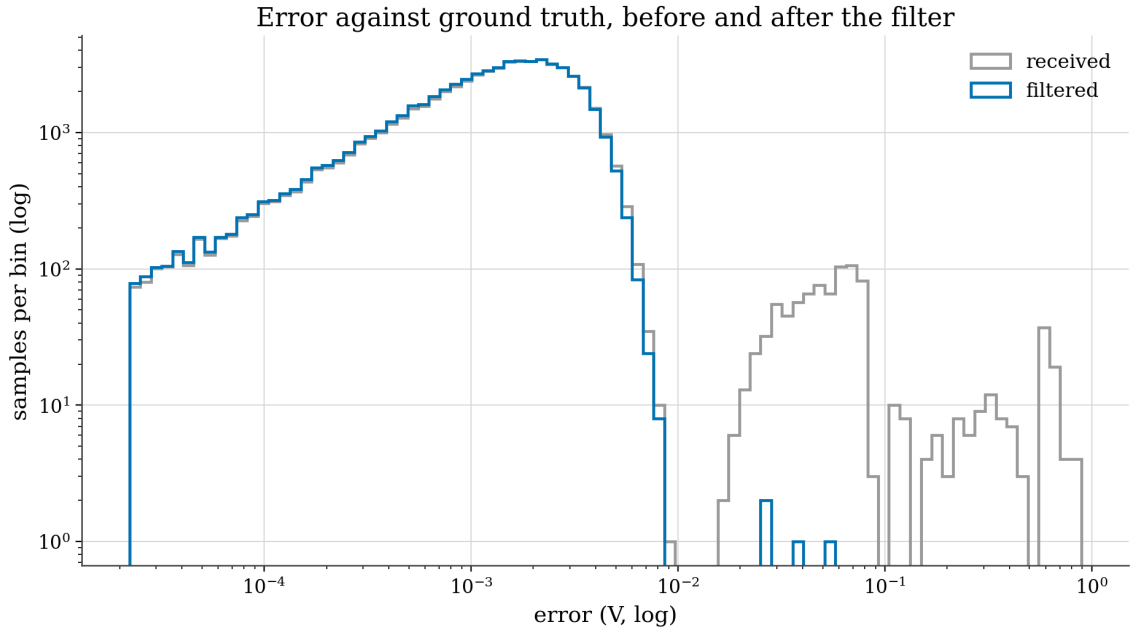


Figure 4.5: Filtering error for the voltage channel.

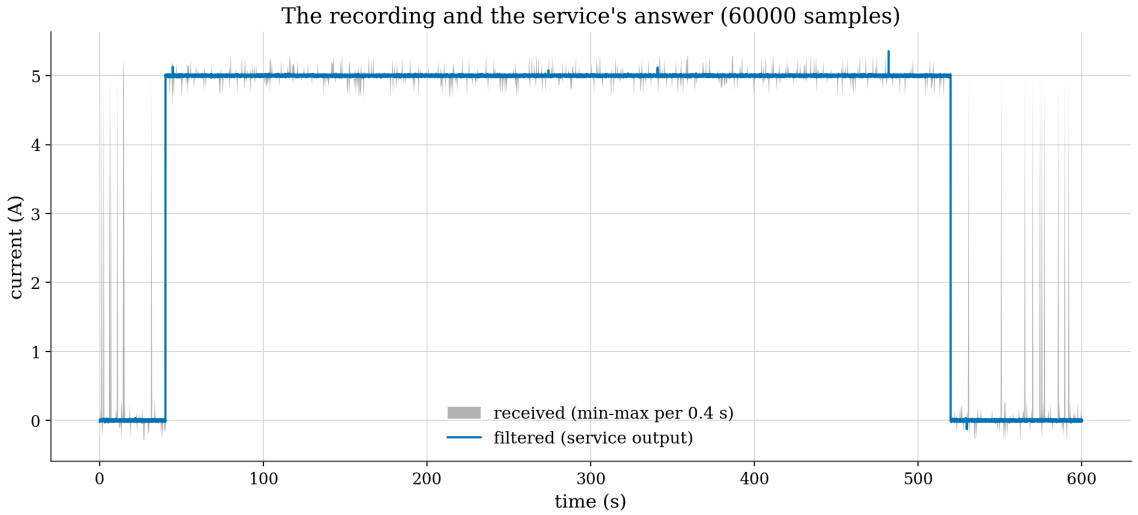


Figure 4.6: Raw and filtered current signals using the rolling Hampel filter.

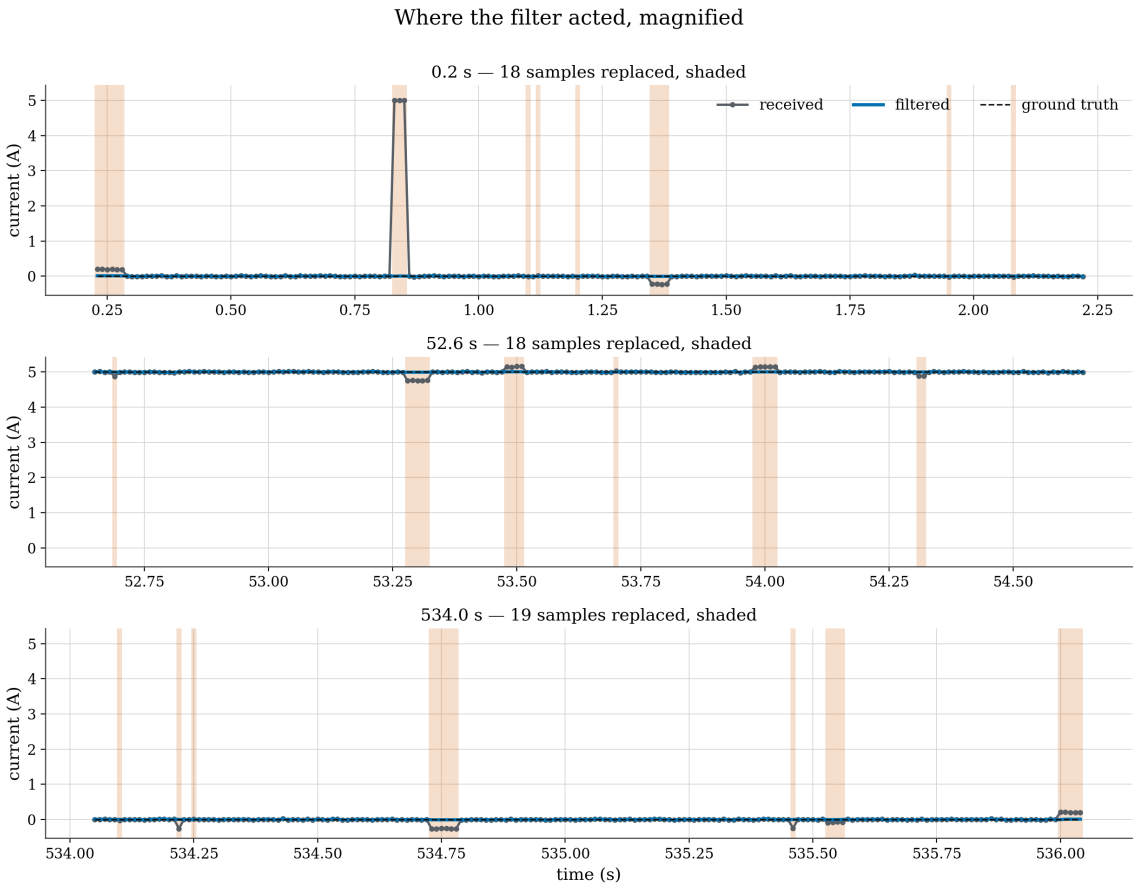


Figure 4.7: Zoomed view of the raw and filtered current signals, highlighting the local effect of Hampel filtering.

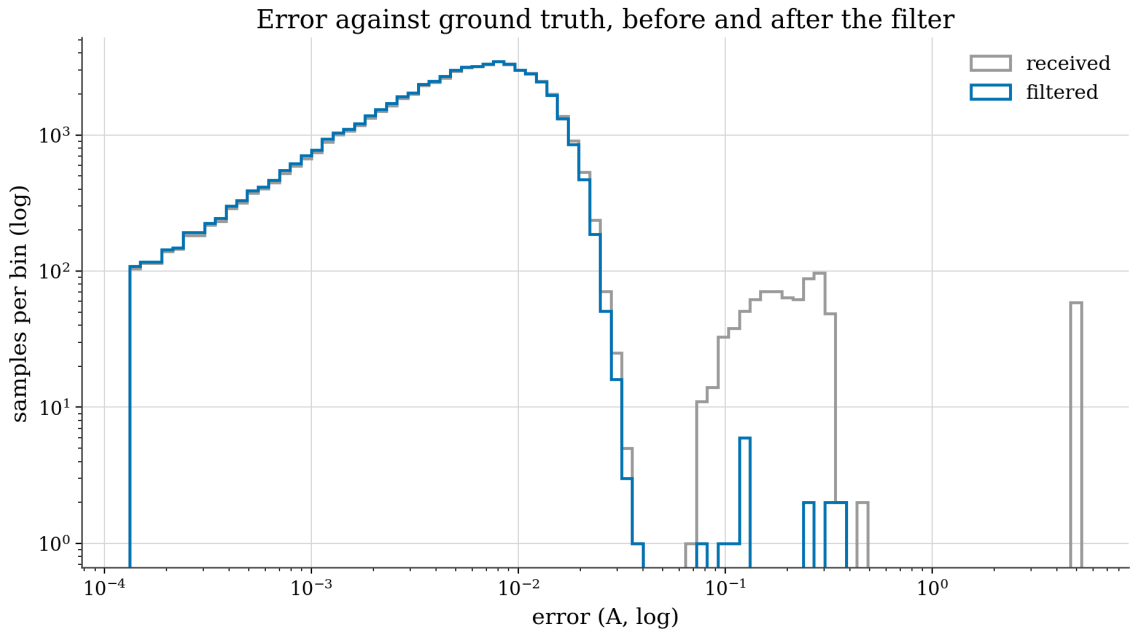


Figure 4.8: Filtering error for the current channel.

4.3.3 Temperature Channel

The temperature-channel results illustrate how the filter affects the temperature trend and local variations. The full overlay and zoomed view provide complementary visual perspectives, while the error plot helps assess the changes introduced by filtering. These observations should be considered together with the temperature recall and RMSE measurements.

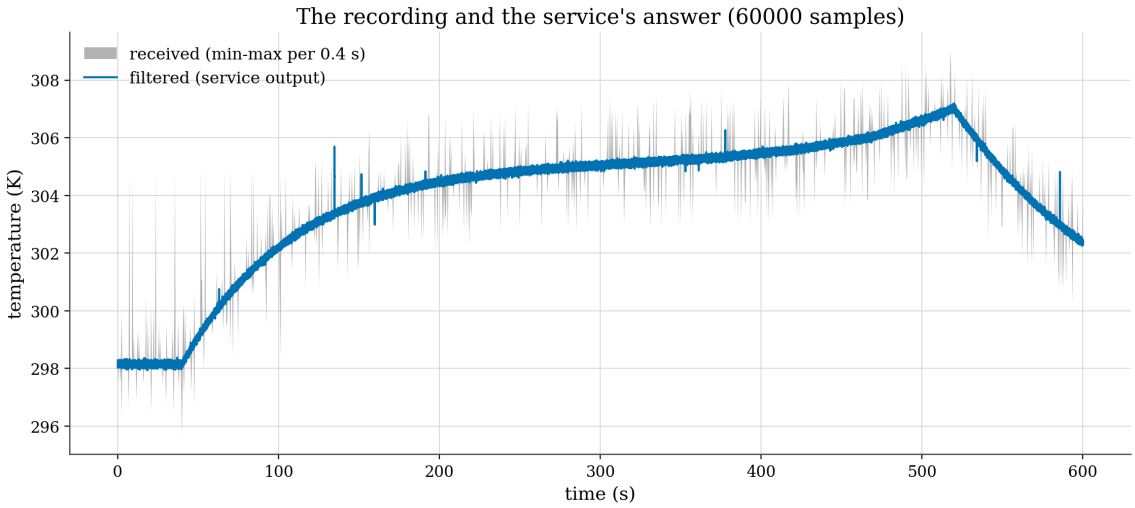


Figure 4.9: Raw and filtered temperature signals using the rolling Hampel filter.

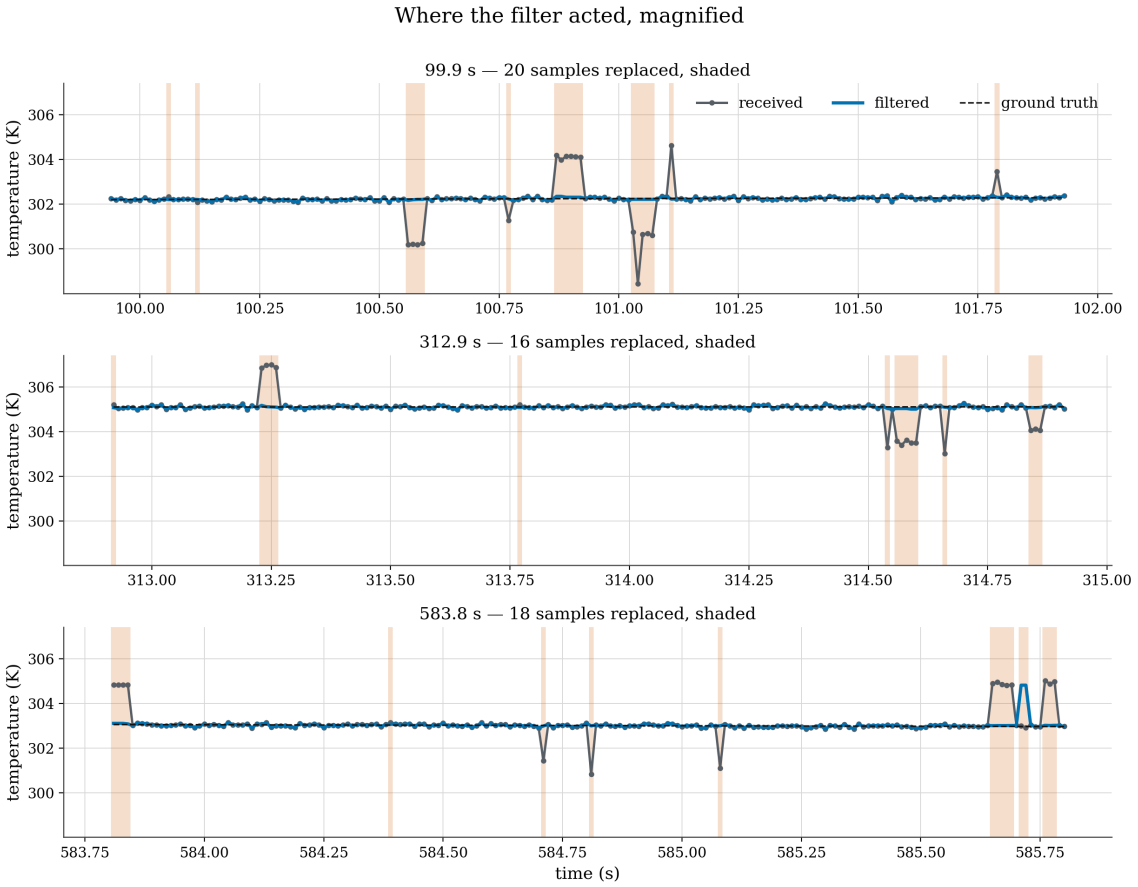


Figure 4.10: Zoomed view of the raw and filtered temperature signals, highlighting the local effect of Hampel filtering.

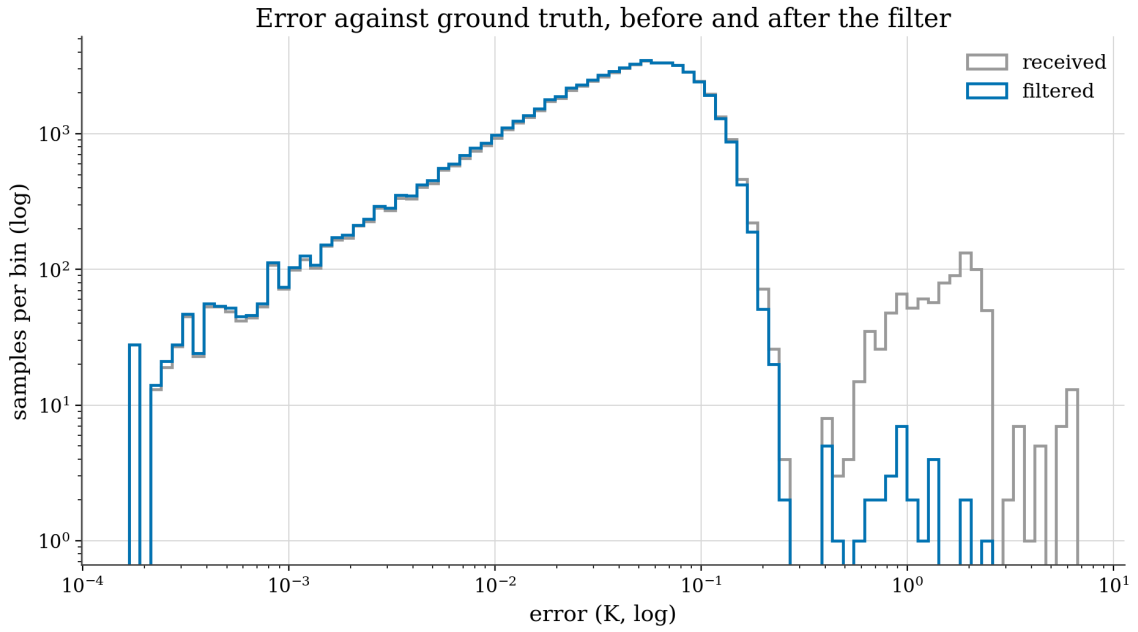


Figure 4.11: Filtering error for the temperature channel.

4.4 Processing Cost and Timing Analysis

The computational cost of the filter was measured using the server's per-block timing instrumentation. Each full block contains 16,384 single-precision floating-point samples, corresponding to 65,536 bytes (64 KiB). The reported timings separate sorting, filtering and total server-side processing.

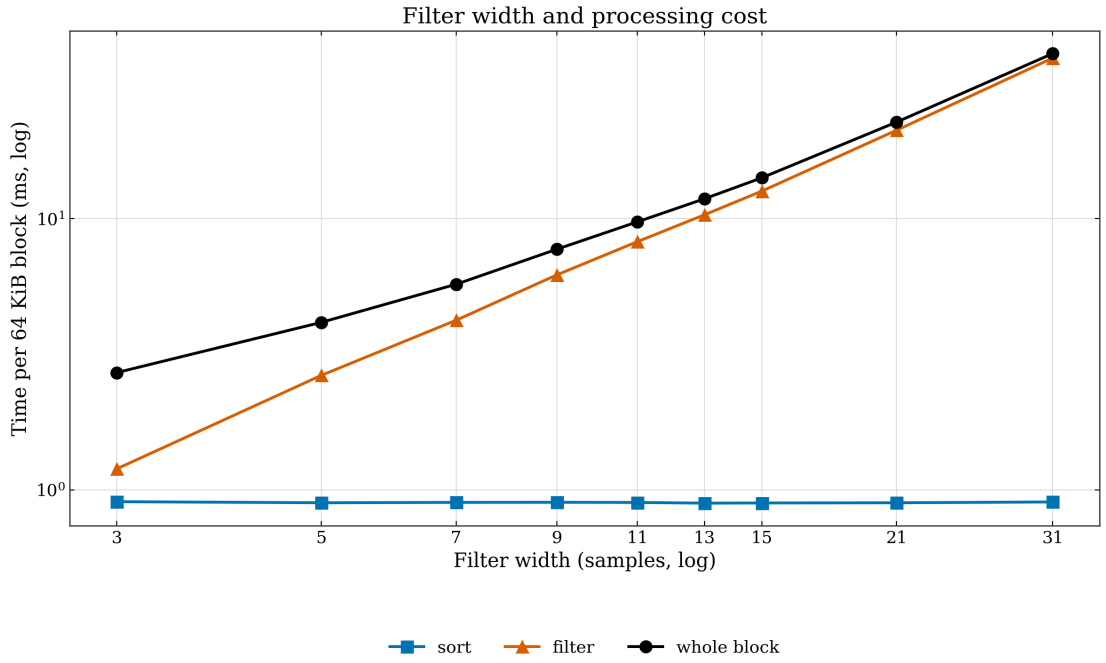


Figure 4.12: Measured sorting, filtering and total server processing time as a function of rolling-window width.

The measurements show that filtering becomes more expensive as the window width increases. The median and MAD must be evaluated for every sample, and the local window operations grow with the number of samples considered. Consequently, a wider window improves the ability to detect longer abnormal bursts but increases processing latency.

The trade-off plot helps select a practical operating width rather than maximizing outlier recall alone. The default width of 9 provides a configurable starting point for balancing signal cleaning and processing cost. The final choice depends on the tolerated latency and the duration of the abnormal events that the application must detect.

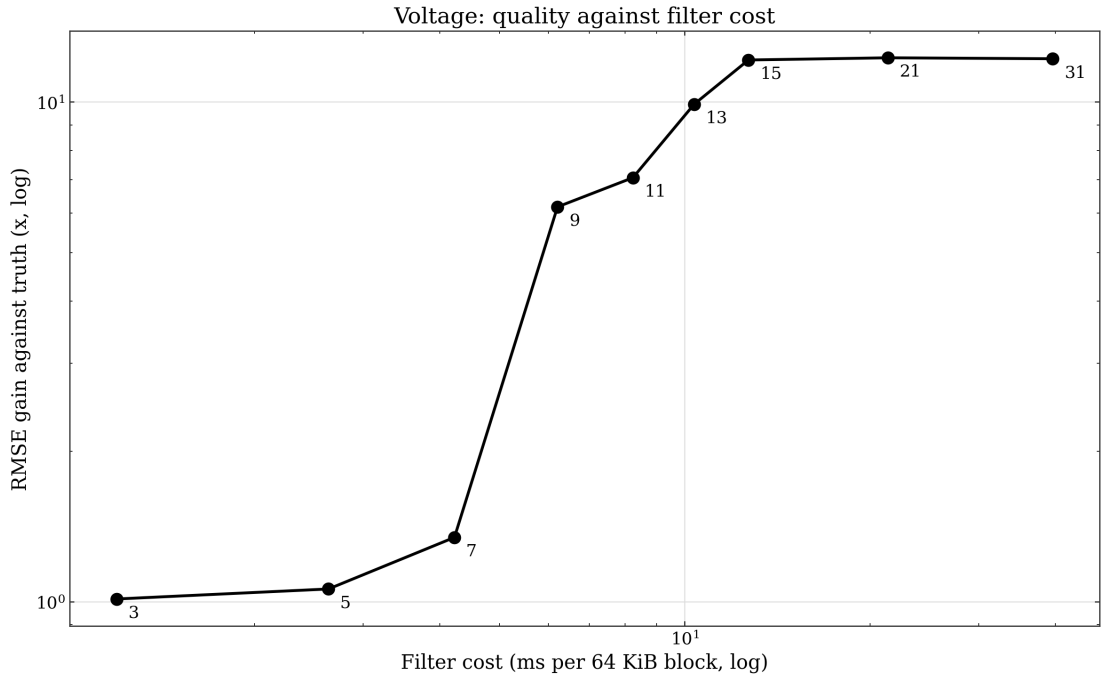


Figure 4.13: Trade-off between filtering quality and computational cost for the voltage channel.

4.5 Parallel Sorting Performance

The parallel merge-sort was evaluated on the Raspberry Pi 4 using one, two, three and four worker threads. The filter width was held constant at its default value of 9 so that the effect of worker count on sorting performance could be examined independently.

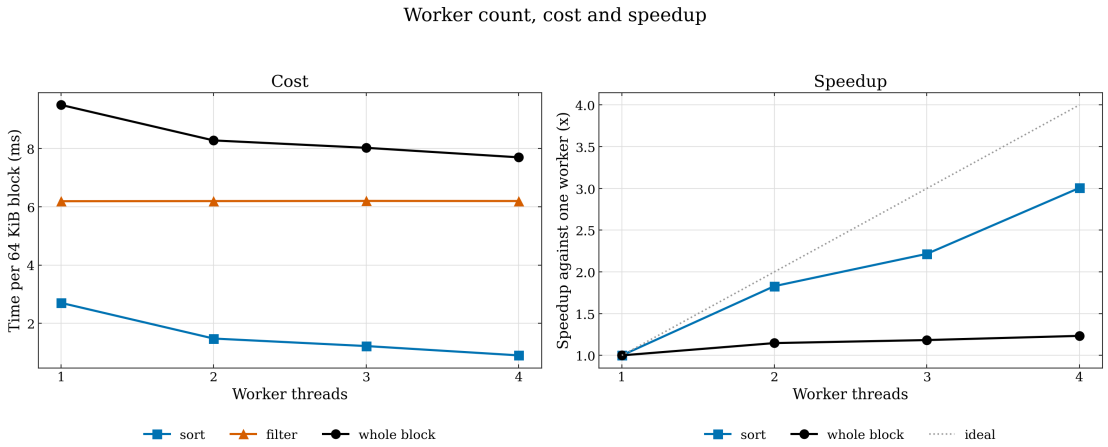


Figure 4.14: Measured processing cost and speedup as the number of merge-sort worker threads increases.

Increasing the number of workers can reduce the sorting time by allowing independent portions of the data to be processed concurrently. However, the overall processing time also depends on merge-phase synchronization, the rolling filter and other server operations. There-

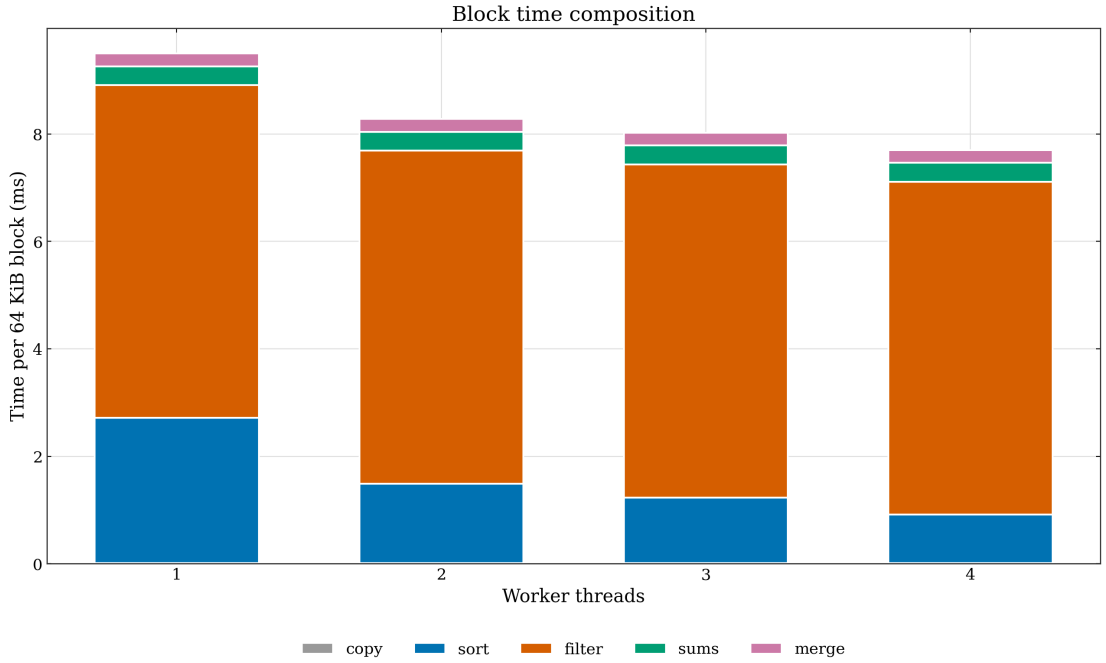


Figure 4.15: Distribution of processing time across pipeline stages for different merge-sort worker counts.

fore, improvements in sorting time do not necessarily translate into an equivalent reduction in end-to-end latency.

The worker-count results should be interpreted using both figures: the scaling plot summarizes the change in performance, while the stage breakdown identifies which operations continue to contribute to total processing time.

4.6 System Overhead and IPC

4.6.1 QNX Board Execution and Validation

The complete application was executed on the Raspberry Pi 4 running QNX Neutrino RTOS. The board tests exercised the protocol, virtual sensor, block loader, merge-sort implementation, median filter and shared-memory subsystem. Additional scripts tested client connection retry behaviour and repeated pipeline execution.

The execution evidence demonstrates that the application was run on the target platform rather than being validated exclusively through host-side compilation. The functional tests were used to check data integrity, filtering behaviour, sorting correctness and shared-memory access.

The median filter suite checks the implementation against a reference filter and exercises boundary conditions, abnormal sample patterns, block transitions and invalid arguments. The

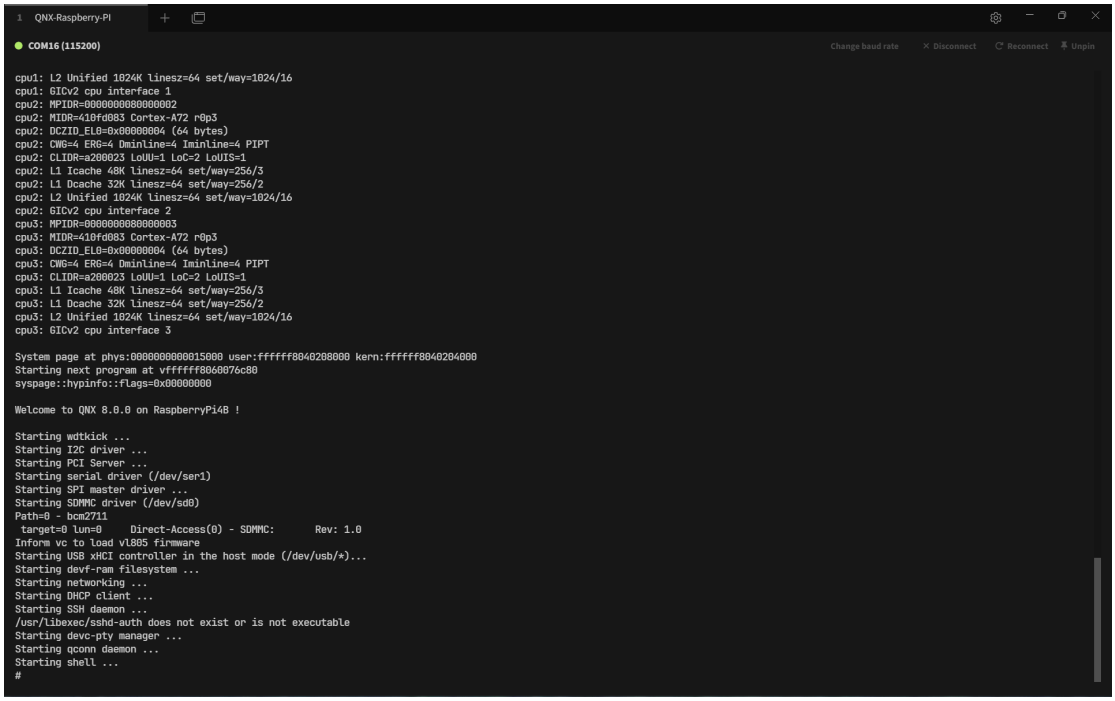


Figure 4.16: Boot evidence from the QNX target environment.

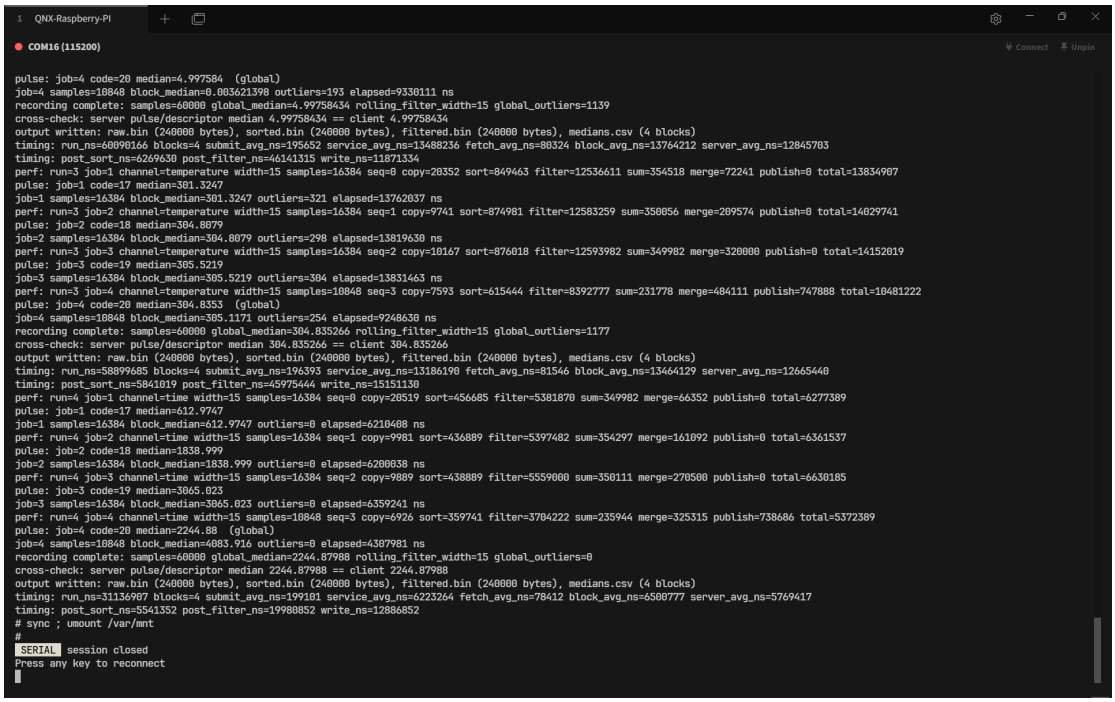


Figure 4.17: Completion of the processing pipeline on the QNX target.

test results provide evidence of functional correctness for the tested cases.

Table 4.2: Measured average timing of the QNX processing pipeline for the voltage channel at filter width 15.

Operation	Average time (ms)
Block submission	0.194
Server service	13.172
Result fetch	0.084
Total block time	13.450

Table 4.2 shows that server-side processing accounts for most of the measured block time in this configuration. Submission and result retrieval contribute comparatively little overhead. These measurements apply to the tested voltage-channel configuration and should not be interpreted as universal execution times for every filter width or input channel.

Chapter 5

Conclusion and Scope for Further Research

5.1 Practical Industrial Applications

The techniques used in this project, including parallel sorting, outlier filtering, and asynchronous IPC, are relevant to several embedded systems applications. The combination of these methods allows large data blocks to be processed efficiently while keeping communication overhead manageable. Some potential applications are discussed below.

- **Battery Management Systems (BMS):** Battery monitoring systems collect voltage, current, and temperature measurements that may contain sudden spikes due to electrical switching or sensor noise. A rolling Hampel filter can help identify and remove these outliers, while sorting the measurements can support the analysis of cell voltage distributions and the identification of cells that may require further inspection.

- **Industrial Edge Gateways:** Industrial systems continuously collect data from vibration, temperature, and other sensors. Processing these measurements in fixed-size blocks using a worker-thread pool can improve throughput and reduce the processing burden on the main application thread. This helps separate data processing from time-sensitive monitoring and control tasks.

- **Automotive Sensor Processing (ADAS):** Advanced Driver Assistance Systems use measurements from sources such as radar, wheel-speed sensors, and other vehicle sensors. Filtering outliers and calculating statistics such as the median can help improve the quality of sensor data used by downstream processing. These operations may be implemented on real-time operating systems such as QNX, where predictable response times are important.

- **Flight and Railway Recorders:** These systems collect continuous streams of operational and sensor data for monitoring, analysis, and later investigation. Processing data in blocks, calculating checksums, and storing results in shared memory can support efficient data handling and integrity checks before the results are archived.

5.2 Conclusion

The project successfully implemented a real-time client-server architecture on QNX. The system demonstrated effective parallel sorting using POSIX threads and robust outlier removal using a Hampel filter. The use of QNX pulses and shared memory provided an efficient, low-latency method for data transfer and asynchronous notification.

5.3 Scope for Further Research

Future work could include:

- Integrating a live hardware sensor (e.g., via I2C or SPI) instead of a virtual file-based sensor.
- Implementing dynamic thread allocation based on the current system load.
- Exploring alternative sorting algorithms (e.g., Radix Sort) for floating-point data.
- Adding a graphical user interface (GUI) for live monitoring of the metrics.